

RESEARCH

Open Access



A federated and transfer learning framework for high-accuracy, explainable intrusion detection in next-generation data centers

Mbarek Marwan^{1*}, Mohammed Chemmakha¹, Abdelkarim Ait Temghart² and Mohamed Lazaar¹

*Correspondence:

Mbarek Marwan
mbarek_marwan@um5.ac.ma
¹ENSIAS, Mohammed V University
in Rabat, Rabat, Morocco

²Faculty of Sciences, Ibn Tofail
University, Kenitra, Morocco

Abstract

Cybersecurity is a critical concern for next-generation data centers, where fast and reliable Intrusion Detection Systems (IDS) are essential. While traditional deep learning models deliver strong performance, they often face limitations related to data privacy, latency, and dependence on centralized architectures. Accordingly, we propose FedTran, a novel IDS pipeline that supports collaborative learning by leveraging secure model updates and shared learned representations, while rigorously ensuring privacy preservation and data confidentiality. First, data imbalance is handled using SMOTE–Tomek Links, followed by feature selection through metaheuristic optimization techniques, specifically the Growth Optimizer Algorithm (GOA), Whale Optimization Algorithm (WOA), and Particle Swarm Optimization (PSO). Second, FedTran adopts a kappa-weighted fusion approach to integrate pretrained ResNet50, InceptionV3, and DenseNet121 models within a transfer learning (TL) framework. By mapping tabular features to structured RGB images, the proposed IDS model facilitates hierarchical and spatial feature learning using CNN-based architectures. Third, Federated Averaging (FedAvg) facilitates decentralized, privacy-preserving collaborative training across distributed edge nodes. In this setting, improved generalization and reduced overfitting are achieved through complementary mechanisms, including data distribution handling, regularization techniques, early stopping, and ensemble learning, thereby limiting sensitivity to dataset-specific noise. On the UNSW-NB15 dataset, the proposed method achieves 96.06% accuracy with FedAvg and 99.98% accuracy with the TL model, significantly surpassing the conventional CNN in both predictive accuracy and training efficiency. Furthermore, the integration of a SHAP-based XAI module allows FedTran to maintain high detection accuracy while offering transparent and interpretable insights for effective security decision-making.

Keywords Intrusion detection systems, Convolutional neural network, Ensemble learning, Feature selection, Tomek links, SMOTE, Federated learning, Transfer learning, Explainable AI, Local shapley additive explanations, Permutation feature importance



1 Introduction

Edge data centers, a distributed IT architecture that brings processing closer to the data source, reduce latency and lower bandwidth costs, thereby improving user experience and application performance [1]. In this setting, typical hyperscale clouds are primarily designed to process essential tasks and store data for long-term retention. By contrast, edge nodes are geographically positioned close to enterprises, or even deployed on-site, to support ubiquitous IoT and 5G adoption [2]. Moreover, edge data centers serve as a viable extension to the currently dominant cloud platform by providing distributed computing capabilities and an open IT architecture. Resource-constrained devices, such as sensors and actuators, can benefit greatly from this model by connecting wirelessly to nearby networks, allowing them to collect and exchange data efficiently [3]. Although existing cloud–edge architectures are capable of meeting next-generation network (NGN) capacity and performance requirements, they continue to face several challenges, including data security and privacy protection, bandwidth limitations, and cache management constraints [4]. Besides, the major vulnerabilities and numerous potential cybersecurity threats have increased due to the rising connectivity and accessibility of susceptible smart devices. Specifically, real-time and low-latency processing of IoT data necessitates the use of lightweight privacy-preservation systems for both authentication and encryption techniques [5]. In light of this fact, it is of utmost importance to build appropriate security measures that accurately identify security vulnerabilities and mitigate them. Unfortunately, the lack of fundamental security protections allows cyber-attackers to launch different kinds of attacks, such as denial of service (DoS), transmission control protocol (TCP) floods, IP spoofing, SYN flooding, buffer overflow, and ping of death [6]. Basically, host-based IDS models are used to monitor system activities for signs of suspicious behavior, whereas network-based IDS models are designed solely to analyze network traffic [7]. To achieve effective detection capability, a hybrid IDS is used, combining a model that detects deviations from the system's normal profile with another that compares incoming patterns against recorded signatures.

Recent research has explored the potential of deep learning (DL) methods to develop IDS models capable of distinguishing between various attack types and normal network traffic. In general, DL models require large amounts of labeled training data to achieve strong performance and avoid overfitting. Federated learning (FL) and transfer learning (TL) provide effective mechanisms to overcome these constraints by supporting cross-domain knowledge transfer and privacy-preserving collaboration among edge devices without raw data exchange [8]. Importantly, edge nodes store their data locally to preserve privacy, instead of relying on centralized techniques that require large amounts of data and incur significant computational overhead. Accordingly, communication among edge nodes is coordinated through the cloud, which distributes the required model parameters and subsequently receives the updated parameters from each node. These updates are then aggregated by the central cloud to construct the global IDS model.

Based on these considerations, by leveraging well-known CNN models that are already trained on datasets stored in the cloud, transfer learning (TL) enables the reuse of these pre-trained models and allows them to be fine-tuned using a smaller amount of labeled data at the edge nodes. The basic idea of federated learning (FL) is to send only local model parameters to the central cloud server while allowing individual edge nodes to train IDS models on limited local datasets [9]. In an FL model, none of the edge

nodes shares its raw data with the cloud. Figure 1 presents the overall framework of the proposed IDS model and illustrates how transfer learning (TL) and federated learning (FL) are jointly integrated to secure edge–cloud data centers, as well as how data are exchanged between the edge nodes and the central backbone cloud system.

By adopting a decentralized IDS architecture, TL and FL techniques enable scalable model training across distributed entities, fostering collaborative intelligence while reducing both latency and power consumption. To keep pace with evolving security threats, a group of participating edge nodes locally processes its stored data and exchanges model updates obtained during the training phase. Under these circumstances, we can develop deep learning models based on TL or FL to share knowledge across edge nodes and enhance accuracy, security and efficiency.

However, intrusion detection system (IDS) models often operate as black boxes, providing limited insight into the rationale behind their AI-driven decisions. Explainable artificial intelligence (XAI) addresses this limitation by providing transparency and interpretability for machine learning predictions [10]. In this context, we apply Shapley Additive exPlanations (SHAP) and Permutation Feature Importance (PFI) to analyze the influence of individual dataset features on prediction outcomes, thereby improving the interpretability and explainability of the proposed IDS behavior. This helps overcome key limitations of AI-based IDSs and provides insight into classification results. In addition, XAI techniques support effective hyperparameter tuning, including the selection of network depth, number of neurons, activation functions, optimizers, learning rate, batch size, and training epochs. This capability significantly reduces training time while improving overall model performance.

The major contributions of this article are as follows:

- (1) We use a hybrid technique for data augmentation and feature selection allows the strengths of both approaches to be integrated into a single efficient model. Such preprocessing for IDS classification is intended to improve model performance by enhancing data quality and reducing both noise and training time.
- (2) We propose an efficient transfer learning (TL) algorithm based on CNN model for an IDS model, in which models are first trained on large datasets from related domains and then fine-tuned on the target dataset to address the problem of data scarcity. Moreover, an ensemble TL approach is employed to improve deep learning performance by combining multiple models.

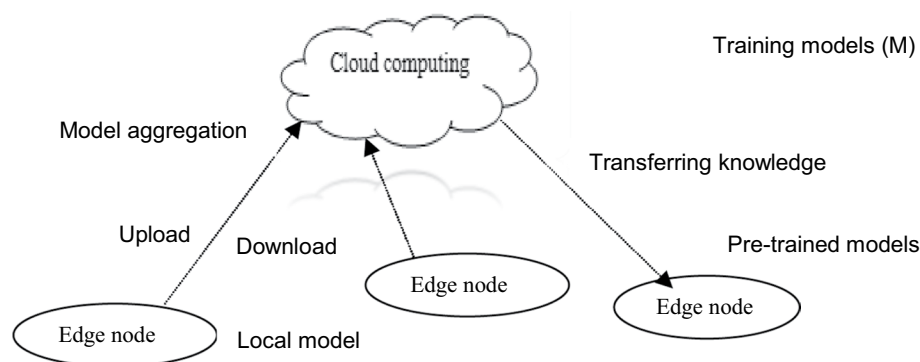


Fig. 1 Architecture of an IDS based on TL and FL in edge–cloud data centers

- (3) We employ a distributed machine learning technique that allows multiple edge nodes to collaborate in developing a shared model. Without relying on centralized training data, federated learning, based on a collaborative machine learning approach, can detect cyberattacks and safeguard sensitive information in cloud–edge data centers.
- (4) We explain and validate the proposed approach by employing both SHAP and PFI methods to enhance IDS transparency. This also provides an effective way to tune and optimize the hyperparameters of the proposed IDS model.

The remainder of this article is organized as follows. Section 2 reviews the related work. Sections 3 and 4 present the background of the proposed framework for improving IDS in edge data centers, including transfer learning, federated learning, and the XAI model. Section 5 is devoted to the simulation experiments and the analysis of the corresponding results. Moreover, we provide insights into how each feature influences the proposed IDS model using XAI. Finally, conclusions and future work are presented in Sect. 6.

2 Related work

As IoT devices have grown rapidly, security issues at the different layers of communication between these devices and edge data centers need to be addressed. To this aim, recent research has focused on deep learning-based intrusion detection systems to minimize cybersecurity risks and protect against cyber threats. Among deep learning neural networks, several CNN-based methods are employed in IDS systems to protect against Zero-Day attacks. In [11], mean-convolutional layer and SoftMax classifier were used to learn and detect abnormal content features. To achieve a global optimum, the combination of CNN and recurrent neural networks (RNN) is an efficient approach to build an IDS model with high detection capability [12]. The IDS system developed by Zhang et al. [13] was based on the GoogLeNet Inception and CNN models. Yin et al. [14] combined an enhanced deep capsule network with an attention mechanism to develop an IDS model that improves feature representation and accurately distinguishes between normal and attack events. After using SMOTE to handle an imbalanced dataset, the authors in Lin et al. [15] relied on an RNN and a long short-term memory (LSTM) network to detect various types of attacks with high accuracy and efficiency. In the same line, the authors in Shone et al. [16] used a deep autoencoder and random forest for feature extraction. Next, CNN-based multi-class classification is used for network traffic data. In Li et al. [17], the authors suggested deep transfer learning (DTL) to develop the IDS model on CIC-IDS2017 for achieving better performance. Okey et al. [18] suggested a framework for intrusion detection in cloud-based IoT systems. The proposal uses transfer learning to support devices with limited resources. In most cases, tabular datasets are not suitable for modeling IDS-based CNNs as they do not consider spatial relationships between features. In Li et al. [19], the authors proposed a graphic conversion technique that converts attack data into image format for the NSL-KDD dataset. In a similar vein, Zhu et al. [20] proposed an optimized approach that maps dataset features to pixel positions in an output image by grouping similar features. To better adapt the base models to edge data center environments, the hyperparameters of the proposed IDS model are carefully tuned and optimized to further enhance performance.

Recently, hybrid blockchain-based IDS have emerged as a new decentralized approach that performs security analysis and access control close to the data source, thereby reducing latency and enhancing user privacy [21, 22]. Within this framework, machine

learning is employed to enhance intrusion detection performance, while blockchain technology provides secure, immutable, and decentralized data storage [23]. Moreover, the platform employs the InterPlanetary File System (IPFS) for off-chain data storage and utilizes zero-knowledge proof (ZKP) mechanisms to ensure data integrity and secure data transmission [24]. On the one hand, the Practical Byzantine Fault Tolerance (PBFT) consensus algorithm ensures system resilience and prevents single points of failure, enabling effective intrusion detection within heterogeneous IoT networks [25]. On the other hand, the proposed hybrid blockchain-based framework integrates elliptic curve cryptography (ECC) with federated learning to balance robust security guarantees against the limited computational and energy resources of IoT devices [26].

Nevertheless, despite significant advances in deep learning-based intrusion detection for edge data centers, numerous challenges remain to be addressed. This study introduces an advanced hybrid model that integrates transfer learning (TL) and federated learning (FL) to enhance the accuracy and robustness of the IDS while addressing the complexity and challenges of resource-constrained edge networks. Overall, the combination of TL and FL models promotes strong generalization capabilities, making the proposed framework suitable for deployment across diverse network environments and intrusion detection benchmarks beyond UNSW-NB15. In fact, the FL model further enhances generalization by exposing the global model to heterogeneous data distributions across multiple clients or data center nodes, while the TL model enables learning of generic, dataset-agnostic network traffic representations, such as flow-level statistics and temporal patterns. On the other hand, explainable artificial intelligence (XAI) provides a principled way to guide the selection and configuration of hyperparameters during the model design process. Wang et al. [27] employed SHAP and Local Interpretable Model-agnostic Explanations (LIME) to integrate both global and local explanations. In this context, XAI techniques are used to identify and interpret the relationships between different types of data center attacks and the corresponding dataset features.

3 Proposed framework

Recently, advanced deep learning (DL) techniques have been proposed for IDS in edge computing to overcome the limitations of traditional AI techniques due to the increased number of learning layers. To discover hidden trends and patterns in raw datasets, DL models perform better in the case of large datasets, as their performance is directly proportional to the volume of training data. On the one hand, transfer learning (TL) is an effective approach to model and predict different types of cyberattacks in edge nodes with a scarcity of data and computing resources by using models that have been previously trained on cloud servers. This study examines the effectiveness of deep learning approaches in improving intrusion detection through the identification of unknown attacks in data centers. To validate the proposed IDS model, we assessed its effectiveness in correctly identifying various attack categories based on multiple performance metrics. Our research suggests a novel approach that leverages both transfer learning and convolutional neural networks (CNNs) to build intelligent IDS in edge data centers. Because of its excellent performance in classification, CNN is not only a very strong approach for handling numeric tabular data, but it can also handle image data. In fact, CNNs make use of filters (also known as kernels) to perform operations on images, including edge detection, blur, and sharpening. To achieve this, we first need to transform the

numerical tabular data into an image representation. The resulting image labels have to be utilized in accordance with the dataset's chunk attack patterns. Importantly, this pre-trained model combines the concepts of transfer-learned architectures that utilize the strengths of various models such as OnlyCNN, VGG16, DenseNet121, ResNet-50, EfficientNetB0, InceptionV3, and MobileNetV3Small to train the IDS model. On the other hand, FL model has been considered a viable option to address cyberattacks as it enables several edge nodes to train a common IDS model without actually exchanging local data with other nodes. This technique requires model updates to be sent to the cloud servers for the aggregation of local model parameters.

Unlike existing works, the proposed framework applies an end-to-end explainable AI (XAI) approach tailored to produce reasonable and intuitive explanations of CNN-based IDS models. On the basis of the above considerations, Fig. 2 presents an overview of the proposed IDS model with the XAI module.

The proposed framework consists of four key components: (1) the input dataset, which consists of both normal and malicious network traffic; (2) a distributed machine learning framework based on FL model; (3) a TL model that relies on pre-trained AI models to improve performance; and (4) XAI tools to interpret and explain IDS models. Overall, the modular design allows efficient fine-tuning using limited labeled data, enabling rapid adaptation without full retraining. As a result, the proposed framework is expected to preserve stable performance across different intrusion datasets despite domain shifts. Moreover, transfer and federated learning jointly encourage generalized representations and robustness to dataset-specific bias, thereby reducing the risk of overfitting to dataset-specific patterns.

3.1 Data preprocessing

An essential part of the data preprocessing workflow is data cleansing. This process entails locating, rectifying, or eliminating dataset issues, including handling outliers, deleting duplicates, and dealing with inconsistent or missing data. In this instance, we converted categorical attributes to numerical values using one-hot encoding. Moreover, we use an unsupervised technique (called an isolation forest) that employs a group of decision trees to find outliers in a dataset. Given the limited availability of training data

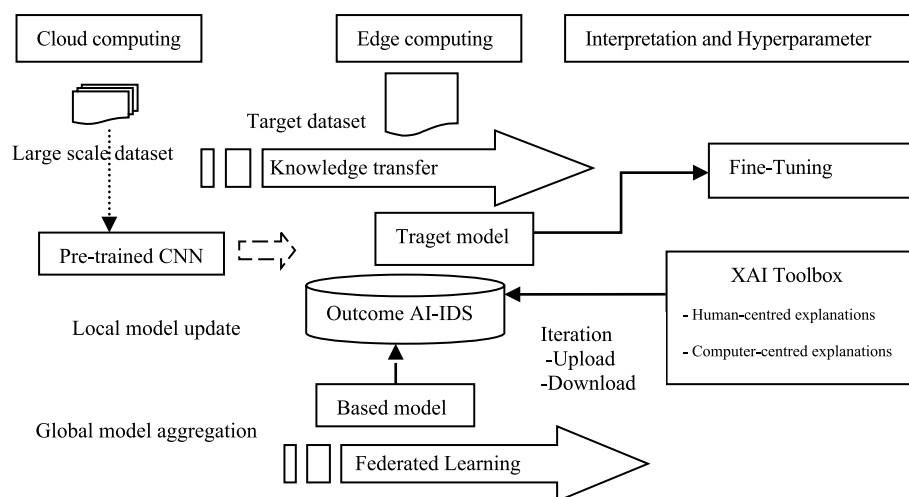


Fig. 2 Schematic diagram for TL and FL in edge data centers

for deep learning models, we propose a hybrid sample generation strategy that creates realistic virtual samples similar to real data to improve classification performance on imbalanced datasets. To enhance IDS performance, nature-inspired metaheuristic algorithms are employed to eliminate irrelevant and redundant features from the dataset. Therefore, reducing the number of features not only lowers computational complexity but also decreases training time, thereby facilitating the model-building process. It has been widely demonstrated that data normalization enhances AI model performance and accelerates convergence during training. In this context, several feature-scaling techniques are commonly used, including Z-score normalization, logarithmic normalization, decimal scaling, and Min–Max scaling.

Let $\mathcal{D}$ denote a dataset defined over a feature space $\mathcal{N}$, with $X = \{x_1, \dots, x_r\}$, where $1 < r \leq n$ and $|\mathcal{N}| = n$ represents the total number of instances in the dataset. In this setting, Algorithm 1 is applied to perform a linear transformation on the original data, resulting in the normalized dataset denoted as X_{norm} .

Input : $X(x_1, \dots, x_r)$, where $1 < r \leq n$
Output : $X_{\text{norm}}(x_1^{\text{norm}}, \dots, x_r^{\text{norm}})$

```

1:: For i=1 to n do
2:   If ( $x_i$  a non-numeric data ) then
3:     Step 1: Using Scikit-learn to one-hot encode categorical features
4:     Step 2: Applying MinMaxScaler to scale features to the [0, 1] range
               
$$x_i^{\text{norm}} = \frac{x_i - (x_i)_{\min}}{(x_i)_{\max} - (x_i)_{\min}}$$

5:   End if
6: End for
7: Return  $X_{\text{norm}}$ 

```

Algorithm 1 Pseudo code for min–max scaling

Following data preprocessing, a hybrid sampling strategy that integrates oversampling and undersampling methods is proposed to mitigate class imbalance and reduce dimensionality. Specifically, the Synthetic Minority Oversampling Technique (SMOTE) is employed to cluster the data and generate synthetic minority samples, while the Tomek links method is applied as a post-processing undersampling step to remove noisy and borderline majority instances [28]. SMOTE [29] is applied to oversample minority classes in a manner that mitigates overfitting.

First, a minority class sample X_0 is randomly selected. Then, one sample X_k is randomly chosen from the set of its k -nearest minority neighbors in the Euclidean space $\mathbb{R}^n$. The neighbors are identified using the Euclidean distance between X_0 and the other minority samples, defined as:

$$X_k = \text{dist}(X_0, X_i) = \sqrt{\sum_{j=1}^n (x_j - x_{0j})^2} \quad (1)$$

Subsequently, a new synthetic sample Z is generated by performing linear interpolation between X_0 and X_k , as expressed by:

$$Z = X_0 + \text{rand}(0,1) \times (X_k - X_0), \quad (2)$$

where $\text{rand}(\cdot)$ denotes a uniformly distributed random variable in the interval $[0, 1]$.

For the j -th attribute of a synthetic sample Z^j , we first define the minority and majority sample sets as $S_m = \{s_{m1}, s_{m2}, \dots, s_{ma}\}$ and $S_t = \{s_{M1}, s_{M2}, \dots, s_{Mb}\}$, respectively, where a and b denote the number of minority and majority samples. To rebalance the imbalanced dataset, the distances between the generated synthetic sample Z and the minority samples in S_m , denoted by $D_{\text{minority}}(Z, S_m)$, as well as the distances between Z and the majority samples in S_M , denoted by $D_{\text{majority}}(Z, S_M)$, are computed. These distances are calculated using the Euclidean metric, defined as:

$$D_{\text{minority}}(Z, S_m) = \sqrt{\sum_{j=1}^n (Z^j - s_{mq}^j)^2}, q \in [1, a] \quad (3)$$

$$D_{\text{majority}}(Z, S_M) = \sqrt{\sum_{j=1}^n (Z^j - s_{Mf}^j)^2}, f \in [1, b] \quad (4)$$

Based on these computed distances, the corresponding distance sets for each generated synthetic sample Z are constructed as follows:

$$A_{\text{minority}} = \{D_{\text{minority}}(Z, s_{m1}), D_{\text{minority}}(Z, s_{ma})\} \quad (5)$$

$$A_{\text{majority}} = \{D_{\text{majority}}(Z, s_{M1}), D_{\text{majority}}(Z, s_{Mb})\} \quad (6)$$

To improve class separability and reducing class overlap, the new synthetic instances are accepted if:

$$\min(A_{\text{majority}}) < \min(A_{\text{minority}}) \quad (7)$$

Second, the Tomek Links technique [30] is employed as a post-processing step to eliminate borderline or ambiguous samples that may degrade model performance.

Let $\mathcal{D}_{\text{min}}$ and $\mathcal{D}_{\text{maj}}$ denote the minority and majority classes of the dataset $\mathcal{D}$, respectively:

$$\mathcal{D}_{\text{min}} = \{u_1, u_2 \dots u_{|\mathcal{D}_{\text{min}}|}\}, \text{ and } \mathcal{D}_{\text{maj}} = \{v_1, v_2 \dots v_{|\mathcal{D}_{\text{maj}}|}\}$$

Consider a pair of instances $u_i \in \mathcal{D}_{\text{min}}$ and $v_i \in \mathcal{D}_{\text{maj}}$. The pair (u_i, v_i) is defined as a Tomek link if there exists no third instance $s_t \in \mathcal{D}$ such that:

$$\begin{cases} \text{dist}(u_i, s_t) < \text{dist}(u_i, v_j), \\ \text{dist}(s_t, v_j) < \text{dist}(u_i, v_j), \end{cases} \quad (8)$$

To achieve this, when the Tomek Links method is applied to $\mathcal{D}$, we need to find the nearest instance $v_a \in \mathcal{D}_{\text{maj}}$ for each minority instance $u_i \in \mathcal{D}_{\text{min}}$.

$$v_a = \arg \min_{v \in \mathcal{D}_{\text{maj}}} \text{dist}(u_i, v)$$

The following formula is used to construct the link T_{12} .

$$T_{12} = \{(u_i, v_a) | u_i \in \mathcal{D}_{\text{min}}, v_a \in \mathcal{D}_{\text{maj}}\} \quad (9)$$

Similarly, for each majority instance $v_i \in \mathcal{D}_{\text{maj}}$, its nearest neighbor $u_b \in \mathcal{D}_{\text{min}}$ is determined as:

$$u_b = \arg \min_{u \in \mathcal{D}_{\min}} \text{dist}(u, v_j)$$

These pairs form the set:

$$T_{21} = \{(u_b, v_j) \mid u_b \in \mathcal{D}_{\min}, v_j \in \mathcal{D}_{\max}\} \quad (10)$$

Finally, the complete set of Tomek links is obtained by identifying the mutual nearest-neighbor pairs:

$$T = \{T_{12} \cap T_{21}\} \quad (11)$$

After generating synthetic samples, Tomek Links method is applied to identify and remove instances near the decision boundary, thereby eliminating overlapping or noisy samples and improving class separation within the dataset.

3.2 Feature selection

An essential part of feature selection is the process of choosing a subset $\mathcal{S}$ of the most relevant inputs from the feature set Γ of the dataset $\mathcal{D}$ when building and training an IDS model Ψ . This can be achieved through the following strategy:

$$\arg \max_{\Gamma \subseteq \mathcal{S}} \Psi(\mathcal{S}) \quad (12)$$

w.s.t $\mathcal{S} \ll \mathcal{T}$

where $\mathcal{S}$ and $\mathcal{T}$ are the selected and original features set, respectively.

In this paper, we propose generating an ensemble feature selection set by merging multiple feature subsets obtained from different methods. Specifically, we identify the most influential features in the dataset $\mathcal{D}$ for classifying various attack types using nature-inspired metaheuristic techniques, including Particle Swarm Optimization (PSO), the Growth Optimizer Algorithm (GOA), and the Whale Optimization Algorithm (WOA).

3.2.1 PSO algorithm

PSO algorithm [31] is a metaheuristic optimization technique inspired by the collective foraging and movement behavior of bird flocks and fish schools. Mathematically, PSO is formulated to search for the optimal position of m particles within a $\mathbb{D}$ -dimensional space. In PSO, each particle P_i is represented by the position vector $P_i = (p_{i1}, p_{i2}, \dots, p_{i|\mathbb{D}|})$, which corresponds to a candidate solution in the $\mathbb{D}$ -dimensional search space, and the velocity vector $V_i = (v_{i1}, v_{i2}, \dots, v_{i|\mathbb{D}|})$. Each particle's movement is influenced by its personal-best position, denoted as $pbest_i = (pbest_{i1}, pbest_{i2}, \dots, pbest_{i|\mathbb{D}|})$. In addition, the best position found by the entire swarm is referred to as the global-best position, denoted as, $gbest_i = (gbest_{i1}, gbest_{i2}, \dots, gbest_{i|\mathbb{D}|})$.

In this case, particle P_i moves toward its previous personal-best position $pbest_i$, and the swarm's global-best position $gbest_i$ to seek the optimal solution. Accordingly, each dimension ($1 \leq d \leq |\mathbb{D}|$) is updated using the following equation:

$$\begin{cases} v_{ir}^{t+1} = w v_{ir}^t + \sigma_1 \rho_1 (pbest_{ir}^t - x_{ir}^t) + \sigma_2 \rho_2 (gbest_{ir}^t - x_{ir}^t) \\ p_{ir}^{t+1} = p_{ir}^t + v_{ir}^{t+1} \end{cases} \quad (13)$$

where σ_1 and σ_2 are learning factors, ρ_1 and ρ_2 are random numbers in the range (0,1), w is the inertial weight.

3.2.2 WOA algorithm

WOA method [32] is inspired by the bubble-net hunting strategy of humpback whales to balance global and local search and enhance solution accuracy. This algorithm consists of three operators: exploration (searching for prey), exploitation (bubble-net attack), and identification (encircling prey). In the subsection that follows, we discuss each operator in more detail. During the first step of WOA, the ideal location in the search space is unknown. As a result, the current best solution is treated as the optimal location, or prey. The WOA algorithm is a swarm intelligence optimization method that models the hunting behavior of humpback whales. In general, it begins with a set of random solutions. During the optimization process, the remaining agents adjust their positions according to the best search agent. Accordingly, the encircling phase is defined in Eqs. (14) and (15).

$$\vec{E} = \left| \vec{C} \diamond \vec{X}^*(t) - \vec{X}(t) \right| \quad (14)$$

$$\vec{X}(t+1) = \vec{X}^*(t) - \vec{A} \vec{E} \quad (15)$$

where t is the current iteration number, $\diamond$ is an element-by-element multiplication, $\vec{X}^*(t)$ is the position vector of the best solution.

In this case, $\vec{C}$ and $\vec{A}$ are coefficient vectors, which can be expressed by the following formula:

$$\vec{A} = 2 \vec{a} \cdot \vec{r} - \vec{a} \quad (16)$$

$$\vec{C} = 2 \vec{r} \quad (17)$$

In this instance, the value of $\vec{r}$ is randomly generated in the range $[0, 1]$, while the value of $\vec{a}$ decreases linearly as the number of iterations increases, within the range $[0, 2]$.

During the second phase, known as the exploitation phase (bubble-net attack), the search process is guided by two mechanisms, namely the shrinking encircling mechanism (SEM) and the spiral updating position (SUP). These methods compute the distance between the coordinates of the prey and the whale. Specifically, spiral equations are expressed for the prey and whale positions to mimic the helix-shaped movement of humpback whales, as follows:

$$\vec{E}' = \left| \vec{X}^*(t) - \vec{X}(t) \right| \quad (18)$$

$$\vec{X}'(t+1) = \vec{E}' \cdot e^{\rho} \cdot \cos(2\pi\rho) + \vec{X}^*(t) \quad (19)$$

In this case, ρ is a constant used to specify the logarithmic spiral's shape, and ρ is a random number in $[-1, 1]$. WOA utilized a random parameter (ρ) to choose either the spiral-shaped mechanism or the encirclement mechanism with a 50% probability. The mathematical model is as follows:

$$\vec{X}(t+1) = \begin{cases} \vec{X}^*(t) - \vec{A} \cdot \vec{E} & \text{if } \rho < 0.5 \\ \vec{D}' \cdot e^{i\phi} \cdot \cos(2\pi\rho) + \vec{X}^*(t) & \text{if } \rho \geq 0.5 \end{cases} \quad (20)$$

To find the global optimum, individual whales must travel far from reference whales in the feasible solution space during the exploration (prey-searching) stage. Specifically, for a position vector $\vec{X}_{\text{rand}}$ randomly selected from the current whale population, the mathematical model is expressed as:

$$\vec{D} = |\vec{C} \cdot \vec{X}_{\text{rand}} - \vec{X}| \quad (21)$$

$$\vec{X}(t+1) = \vec{X}_{\text{rand}} - \vec{A} \cdot \vec{E} \quad (22)$$

Note that the WOA algorithm selects between circular and spiral motion strategies based on the value of ρ .

3.2.3 GOA algorithm

GOA algorithm [33] is a powerful metaheuristic method designed to address challenging optimization problems, inspired by human behavior during learning and self-reflection stages.

Let us consider a community in which each individual is represented as $X_i = (x_{i,1}, x_{i,2}, \dots, x_{i,K})$, where $x_{i,k}$ denotes the k^{th} dimensional knowledge of the i^{th} individual. Suppose that m is the total number of solutions in X . The solution for the tested problem is:

$$X_i = \gamma(U - L) + L, \quad i = 1, \dots, m \quad (23)$$

where U and L are the limits of the search domain of the problem, γ is the random value.

In the learning stage, an individual X_i improves by examining the gaps between themselves and other members of the population.

Formally, the gap between the leader and the elite is denoted by Gap_1 ; the gap between the leader and the bottom is denoted by Gap_2 ; the gap between the elite and the bottom is denoted by Gap_3 ; and the gap between two random individuals is denoted by Gap_4 . The following equation represents the mathematical model for each set of gaps:

$$\begin{aligned} \text{Gap}_1 &= x_{\text{best}} - x_{\text{better}} \\ \text{Gap}_2 &= x_{\text{best}} - x_{\text{worse}} \\ \text{Gap}_3 &= x_{\text{better}} - x_{\text{worse}} \\ \text{Gap}_4 &= x_{L_1} - x_{L_2} \end{aligned} \quad (24)$$

where x_{best} is the leader of the society, x_{better} represents elite that is one of the next $P_1 - 1$ best people, x_{worse} is one of the bottom P_1 individuals, x_{L_1} and x_{L_2} are two different random individuals.

P_1 takes the value 5.

Generally, an individual gains more knowledge from Gap_i when the group's gap is larger. To this end, a learning factor (LF) is introduced for the k -th group of knowledge gaps and is modeled as follows:

$$\text{LK}_k = \frac{\|\text{Gap}_k\|}{\sum_{k=1}^4 \|\text{Gap}_k\|}, \quad (k = 1, 2, 3, 4) \quad (25)$$

where LK_k is the k^{th} group knowledge gap normalization.

Based on this, an individual learns more effectively from the k -th knowledge-gap group when LK_k is larger, and vice versa. The extent of knowledge learning varies across individuals, and each individual can evaluate their acquired knowledge using the extent of learning, denoted as SF_i .

$$SF_i = \frac{GR_i}{GR_{\max}}, \quad (26)$$

where GR_i is the growth resistance of the i -th individual.

$GR_{\max}$ is the maximum growth resistance.

In general, individuals with smaller GR_i values tend to focus on local exploitation, whereas those with larger GR_i values are more inclined to perform global search behaviors.

The amount of knowledge that the i^{th} individual gains from the k^{th} group of knowledge gaps is denoted as KA_k . In this case, each x_i can acquire new knowledge from the k^{th} group of knowledge gaps using the knowledge acquisition term KA_k , defined as:

$$KA_k = SF_i \cdot LK_k \cdot Gap_k \quad (27)$$

For iteration t , the i^{th} individual's specialized learning process is given by the following equation:

$$x_i^{t+1} = x_i^t + \sum_{k=1}^4 KA_k \quad (28)$$

The quality of the updated version of each x_i is evaluated and compared with that of the original individual to determine whether a significant improvement has occurred. Individuals whose quality decreases are retained with probability P_2 ; otherwise, they are discarded. This selection procedure is formally described in Eq. (29).

$$x_i^{t+1} = \begin{cases} x_i^{t+1} & \text{if } f(x_i^{t+1}) \leq f(x_i^t) \\ \begin{cases} x_i^{t+1} & \text{if } r_1 \leq P_2 \\ x_i^t & \text{otherwise} \end{cases} & \text{otherwise} \end{cases} \quad (29)$$

In this case, r_1 is a random number, and $P_2 = 0.001$ the probability of retaining an individual whose quality has decreased.

On the other hand, the reflection stage requires individuals to compensate for or overcome their deficiencies. Accordingly, each x_i must identify its weaknesses, make improvements, and retain the knowledge gained. More importantly, it is essential to discard outdated information and relearn systematically when certain deficiencies cannot be corrected. The mathematical model of the reflection process in the GOA model is presented in Eq. (30).

$$x_i^{t+1} = \begin{cases} \begin{cases} x_i = \gamma(U - L) + L & \text{if } r_2 < \xi \\ x_i^t + r_4(R_j - x_i^t) & \text{if } r_3 < P_3 \end{cases} \\ x_i^t \end{cases} \quad (30)$$

where r_2 , r_3 and r_4 are pseudorandom numbers in the range $[0, 1]$, P_3 is the probability of reflection, R_j is the knowledge of the j^{th} aspect of a solution in top $P_1 + 1$.

Let FEs and MaxFEs denote the current number of function evaluations and the maximum allowable number of evaluations, respectively. In this case, ξ is computed using the following formula:

$$\xi = 0.01 + 0.99 \times \left(1 - \frac{\text{FEs}}{\text{MaxFEs}}\right) \quad (31)$$

Since each method exhibits specific strengths and limitations, we propose a fusion-based approach that integrates several methods. In this study, we introduce a novel feature selection framework that combines multiple feature-ranking techniques, namely PSO, GOA, and WOA. The proposed framework aims to identify the most relevant and informative features from the IDS dataset. To eliminate unrepresentative variables, the framework aggregates the top-ranked features generated by the different methods using either a union-based or an intersection-based strategy, denoted as X_u and X_s , respectively. Algorithm 2 presents the pseudocode of the proposed feature selection method.

Input : Dataset $\mathcal{D} = \{x_i\} \in \mathbb{R}^{m \times n}$, metaheuristic algorithm $\{L_1, L_2, L_3\}$
Output : selected features sets $\{X_u, X_s\}$

- 1: **For each** model $L_i = \{L_1, L_2, L_3\}$ **do**
- 2: Initialize the selected feature for model
 $L_i : X^{L_i} \leftarrow \emptyset$
- 3: **Repeat**
- 4: Set best score $B \leftarrow 0$
- 5: Set candidate feature set $R \leftarrow \emptyset$
- 6: **For each** $x_j \in \mathcal{D} \setminus X^{L_i}$ **do**
- 7: Evaluate
- 8: Score = Evaluate $((X^{L_i} \cup \{x_j\}), L_i)$
- 9: **If** Score > B **then**
- 10: $B \leftarrow \text{Score}$
- 11: $R \leftarrow X^{L_i} \cup \{x_j\}$
- 12: **End if**
- 13: **If** $R = \emptyset$ **then**
- 14: Break
- 15: **Else**
- 16: $X^{L_i} \leftarrow R$
- 17: **End if**
- 18: **End for**
- 19: **Until** no further improvement is achieved
- 20: **End for**
- 21: Construct the union feature set:
 $X_u = X^{L_1} \cup X^{L_2} \cup X^{L_3}$
- 22: Construct the intersection feature set:
 $X_s = X^{L_1} \cap X^{L_2} \cap X^{L_3}$
- 23: **Return** $\{X_u, X_s\}$

Algorithm 2 Feature section

4 Proposed deep learning model

This study introduces an innovative framework that integrates transfer learning (TL) and federated learning (FL) within resource-constrained edge computing environments, enabling the effective utilization of data from diverse sources while minimizing computational and communication costs.

4.1 Transfer learning

The proposed TL model leverages pre-trained networks often convolutional neural networks (CNNs) originally trained on large-scale datasets. A typical CNN architecture comprises convolutional layers, pooling layers, and fully connected layers. By employing TL, IDS models can learn generic network-traffic representations that are largely data-set-agnostic. These low- and mid-level features have been shown to transfer effectively across benchmark intrusion detection datasets, which share common protocol behaviors despite differences in attack distributions. Another important consideration when applying TL is that samples must be extracted and converted into grayscale images. Various methods exist for converting tabular data into images; however, all images must be resized to a fixed resolution before being fed into the CNN model. Moreover, instead of relying on a single classifier, we employ ensemble learning based on multiple classifiers to aggregate the outputs of CNN models for improved accuracy. In this work, we propose an effective ensemble-based framework that integrates several pre-trained CNN models with varying depths, including OnlyCNN, VGG16, DenseNet121, ResNet-50, EfficientNetB0, InceptionV3, and MobileNetV3Small. The top three best-performing CNN models are then selected to construct the final TL model.

Practically, the system examines incoming traffic and divides it into two categories: normal and abnormal. First, we use a random search strategy to obtain high-performing hyperparameters and optimize the CNN model under examination to detect the most well-known attacks. To select the best three pre-trained models, it is particularly useful to compare each CNN model's performance when using Adam optimizer, which enhances convergence speed and achieves better loss-function minima. On the other hand, the ensemble learning approach aggregates multiple base models to enhance overall predictive performance and robustness. Motivated by this principle, we integrate three distinct base models into a unified framework, enabling the ensemble to leverage the complementary strengths of each component. This combined model serves as a powerful tool for processing and interpreting the input image data, ultimately leading to more accurate and reliable outcomes.

Without delving into implementation details, a typical architectural representation of the TL-based IDS system consists of several interconnected modules, namely: a preprocessing module, a data transformation module, a pre-trained CNN module, an ensemble CNN module, and an XAI module. Figure 3 conceptually illustrates the proposed framework and highlights how these modules are seamlessly integrated to form the complete system.

More interestingly, we investigate explainable artificial intelligence (XAI) to improve the performance of intrusion detection systems. The framework is evaluated based on its ability to accurately predict attacks using well-known evaluation metrics.

4.1.1 Image-conversion process

For image representation, the Min–Max normalization method is first employed to obtain the gray-value matrix during image generation:

$$X = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \times 255 \quad (32)$$

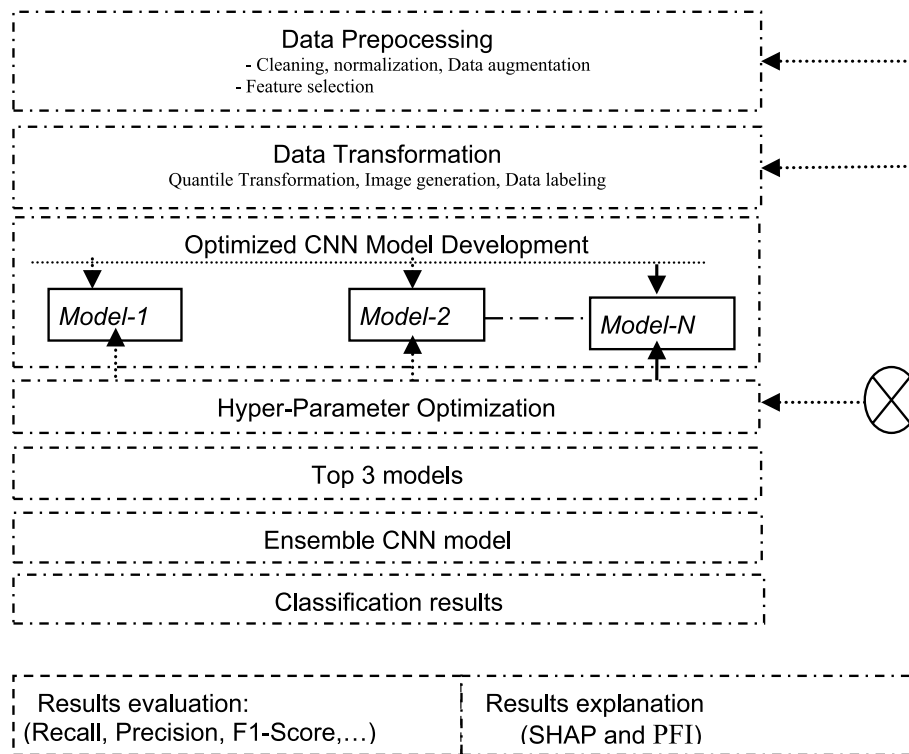


Fig. 3 Transfer learning-based CNN for intrusion detection

To detect specific patterns in the input data, the data samples are converted into chunks of 64 important features. When converted into a 3D image format with three channels, we obtain an image with dimensions $[64 \times 64 \times 64]$. This is accomplished by assigning the first 64 samples of each chunk to the image matrix of channel 1, the next 64 samples to channel 2, and the final 64 samples to channel 3. For this segmentation task, all generated data are mapped into the RGB channels of the output image. In practice, the OpenCV library is used to transform the tabular dataset into an image matrix. In practice, the DeepInsight framework [34] enables the conversion of tabular data into a three-dimensional image format, making it compatible with convolutional neural network (CNN) architectures.

4.1.2 Transfer learning

For feature learning and extraction, three edge-detection filters of size $[3 \times 3 \times 3]$ are applied, resulting in a $[4 \times 4 \times 1]$ image. The classified image is then labeled with the specific attack that appears most frequently within the corresponding chunk. Each dataset chunk consists of $(64 \times 3 = 192)$ consecutive data samples. This procedure is repeated until all labels in the IDS dataset are correctly transformed, thereby improving model performance. A typical design consists of multiple convolutional layers stacked with pooling layers, followed by one or more fully connected layers.

- The rectified linear unit (ReLU) activation function mitigates the vanishing gradient problem during CNN training. ReLU applies the function $\max(0, x)$, which thresholds the input at zero.

$$\text{ReLU}(x) = \{0, x < 0 | 1, x \geq 0\} \quad (33)$$

- *Pooling layers*: These layers are used to reduce the spatial dimensions of feature maps and speed up computation. In this work, we employ the max-pooling technique, which suppresses irrelevant activations and retains only the maximum value within the region of the feature map covered by the filter. The pooling layers use a kernel size of 2×2 with a stride of 2.
- *Global average pooling layer*: This operation reduces the dimensionality of the feature maps by converting the three-dimensional feature stack into a one-dimensional vector.
- *Re-scaling*: Resizing the image to a $256 \times 256 \times 3$ format for the CNN models is the final step of the preprocessing stage.
- *Transfer learning (TL)*: To align the distributions of the source and target domains, we employ the Maximum Mean Discrepancy (MMD) metric, an effective non-parametric criterion. A Reproducing Kernel Hilbert Space (RKHS) is used to estimate the discrepancy between the two distributions.

In this study, we first developed and trained well-established pre-trained models based on the ImageNet dataset [35] and used them as baselines. Specifically, several state-of-the-art pre-trained CNN architectures are employed to train the proposed IDS model, including OnlyCNN, VGG16, DenseNet121, ResNet-50, EfficientNetB0, InceptionV3, and MobileNetV3-Small. These models are pre-trained on the ImageNet dataset, a benchmark for image classification that contains over 14 million images spanning more than 1000 classes.

4.1.3 Stochastic optimization

Deep learning employs hyperparameter optimization algorithms to improve model accuracy automatically. In particular, the optimizer updates the model's weight parameters to minimize the loss function. This function acts as a topographical guide, indicating whether the optimizer is moving in the correct direction toward improved performance by identifying the optimal set of parameters. Common optimizers include Adam, AdaGrad, Nadam, Nesterov Accelerated Gradient (NAG), RMSProp, and SGD, among others [36].

To achieve efficient computation, we adopt the Adam optimizer [37], which requires a limited number of tuning parameters, such as the number of epochs, learning rate, batch size, optimizer type, and number of neurons. The pseudo-code of the Adam algorithm is presented in Algorithm 3.

Input: Initial parameters θ_0 ; Decay rates $\delta_1 \in [0,1)$, $\delta_2 \in [0,1)$
 Small constant ε ; Learning rate γ
 Objective function $\mathcal{J}(\theta)$

Output: Optimized parameters θ_t

- 1: $m_0 \leftarrow 0$; $v_0 \leftarrow 0$; $t \leftarrow 0$
- 2: **While** θ_t does not converge do
- 3: $t \leftarrow t + 1$
- 4: $g_t \leftarrow \nabla \mathcal{J}_t(\theta_{t-1})$
- 5: $m_t \leftarrow \delta_1 m_{t-1} + (1 - \delta_1) \cdot g_t$
- 6: $v_t \leftarrow \delta_2 m_{t-1} + (1 - \delta_2) \cdot g_t^2$
- 7: $\hat{m}_t \leftarrow \frac{m_t}{(1-\delta_1 t)}$; $\hat{v}_t \leftarrow \frac{v_t}{(1-\delta_2 t)}$
- 8: $\theta_t \leftarrow \theta_{t-1} - \gamma \frac{\hat{m}_t}{\varepsilon + \sqrt{\hat{v}_t}}$
- 9: **End while**
- 10: **Return** θ_t

Algorithm 3 Basic Adam optimizer

Accordingly, Adam is used as the optimization algorithm to minimize the objective function with respect to the parameters θ (weights and biases). The algorithm begins by initializing the moving averages to zero ($m_0 = 0$). At the t^{th} iteration, the gradient of the stochastic objective function is denoted by g_t , while m_t and v_t represent the first and second moment estimates (mean and variance) of the gradients, respectively. Moreover, δ_1 and δ_2 are hyperparameters that control the exponential decay rates of these moving averages.

In this study, the Adam optimization algorithm is employed to provide distinct updates for different weights by using an adaptive learning-rate mechanism for stochastic objective functions. This method combines the advantages of momentum-based optimization, AdaGrad, and RMSProp, enabling fast and efficient convergence toward the global minimum of the loss function.

4.1.4 Weighted ensemble model

The proposed IDS model learns the underlying relationships between inputs and outputs, enabling effective attack detection. Let X denote a tree-structured search space and let $f(\cdot)$ be an objective function defined as:

$$f : X \rightarrow \mathbb{R}$$

$$x^* \in \operatorname{agmin}_{x \in X} f(x)$$

In general, when the objective function $f(\cdot)$ is modeled as a Gaussian Process (GP) with distribution $p(x|y)$, the conditional distribution of a sample $\Psi = (x_i, f(x_i))_{i=1}^n$ is also a GP. The Tree-structured Parzen Estimator (TPE) algorithm exploits both distributions $p(y)$ and $p(x|y)$ through two kernel density estimators (KDEs), denoted by $l(x)$ and $g(x)$.

Suppose that the threshold value y^* is selected to represent better optimal values than those predicted by the Gaussian model and is typically set to be lower than the best observed loss, defined as:

$$y^* = \min \{f(x_i) | 1 \leq i \leq n\}$$

Accordingly, the conditional distribution $p(x|y)$ is defined following Bergstra et al. [38] as:

$$p(x|y) \begin{cases} l(x), & \text{if } y < y^* \\ g(x), & \text{if } y \geq y^* \end{cases} \quad (34)$$

where $l(x)$ represents the density estimated from samples x_i for which the $f(x_i) < y^*$. $g(x)$ is the density estimated from the remaining observations.

Note that the threshold y^* is selected as a quantile $\sigma \in (0,1)$ of such that $p(y < y^*) = \sigma$.

For a system comprising $\mathbb{S}$ models, denoted by L_i , the weighted ensemble-averaging deep neural network can be expressed as follows:

$$L_{\text{ensemble}} = \sum_{i=1}^{\mathbb{S}} w_i L_i$$

To this end, we use a softmax function to assign a probability distribution across all models, ensuring that the weights are non-negative and sum to one. This function can be expressed mathematically as follows:

$$w_i = \frac{\exp(\mathbb{J}_i)}{\sum_{j=1}^{\mathbb{S}} \exp(\mathbb{J}_j)},$$

where $\mathbb{J}_i$ is the function that determines the raw weight associated with model L_i .

For simplicity's sake, we assume that $\mathbb{J}_i$ is a logistic regression model constructed for multiple classes with weight w_i . The prediction of model L_i for an input x is expressed as:

$$P(y = i|x) = \frac{e^{x^T w_i}}{\sum_{j=1}^{\mathbb{S}} e^{x^T w_j}}$$

Considering this, we employ an ensemble-based approach to fuse multiple heterogeneous base learners by assigning different levels of importance to each deep CNN model incorporating transfer learning, as illustrated in Fig. 4.

Based on the above considerations, the output of each learner L_i is a probability vector C_K over K classes in the dataset. For each model L_i , a weight α_i is assigned using the softmax function, i.e., $\alpha_i = \text{softmax}(P(y = i | x))$, to generate the prediction for the input sample x . Accordingly, the final output for a given input x_i is:

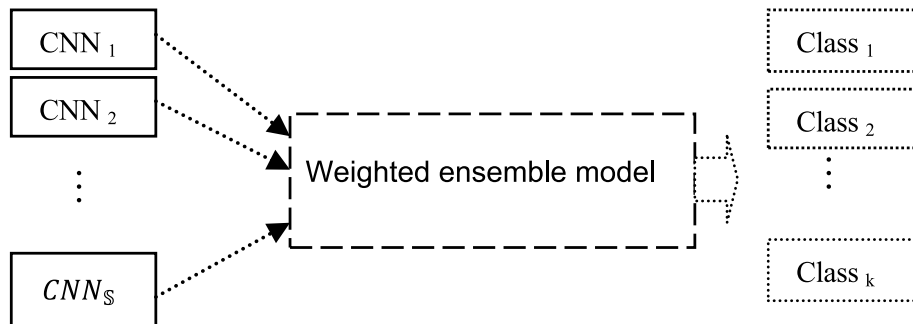


Fig. 4 The proposed weighted ensemble model

$$P = \text{OutputClass}_{x_i, \text{ensemble}} = \arg \max_{t \in \{1, \dots, K\}} \sum_{j=1}^{\mathbb{S}} \alpha_j C_t^{(i,j)}$$

where $\mathbb{S}$ is the number of base learners and $C_t^{(i,j)}$ denotes the predicted probability of class t produced by j -th model for the input x_i .

In this context, Iqbal et al. [39] proposed an accuracy-based weighting scheme that eliminates the least accurate classifier from the ensemble while assigning greater influence to more accurate models in the final decision.

Let acc_j be the accuracy of classifier j and $\mathcal{Q}$ be the index of the least accurate classifier. Then the weight of classifier j is given by:

$$\alpha_j = \text{acc}_j - \mathcal{Q} + 1$$

where $\mathcal{Q} = \underset{j}{\text{argmin}} \text{acc}_j$

In this work, we employ the kappa coefficient, denoted by K_c , to measure the level of agreement between two models and to enhance the credibility of the findings. The kappa coefficient is widely used to assess classification performance by accounting for agreement occurring by chance, and it provides a more reliable evaluation of the final results of the proposed IDS model, particularly in terms of classification accuracy and recall [40].

$$\alpha_i = \ln \left(\frac{1 + Kc_i}{1 - Kc_i} \right)$$

Consider two independent IDS models that classify a dataset with N items into K classes. We frequently use the following arguments:

- n_{ij} is the number of items classified as class i by the first model and as class j by the second model.
- $n_{i.} = \sum_{j=1}^K n_{ij}$ is the number of traffic data samples classified into category i by model 1.
- $n_{.j} = \sum_{i=1}^K n_{ij}$ is the number of traffic data samples classified into category j by model 2.
- $N = \sum_{i=1}^K \sum_{j=1}^K n_{ij}$: is the total number of classified items

As Kappa also takes into account the agreement occurring by chance, the formula for the Kappa coefficient is given in Cohen [41, 42].

$$K_c = \frac{p_a - p_e}{1 - p_e} \quad (35)$$

In this case, $K_c = 1$ indicates perfect agreement, $K_c = 0$ indicates agreement equivalent to chance, and negative values indicate worse-than-chance agreement.

The term p_a denotes the observed agreement, which is expressed as:

$$p_a = \sum_{i=1}^K \frac{n_{ii}}{N}$$

Additionally, p_e represents the chance agreement.

$$p_e = \sum_{i=1}^K \frac{n_{i,i}}{N^2}$$

In this study, the kappa coefficient is calculated using the performance measures, true positive (TP), true negative (TN), false positive (FP), and false negative (FN) (Zhu et al. [43]).

$$Kc = \frac{\mathcal{A} - \mathcal{R}}{1 - \mathcal{R}}$$

In this case, $\mathcal{A}$ is the accuracy, which represents the percentage of correct classifications.

$$\mathcal{A} = \frac{TP + TN}{TP + FP + TN + FN}$$

And $\mathcal{R}$ is another metric that measures the performance of the proposed IDS model, which is defined as follows:

$$\mathcal{R} = \frac{(TP + FP)(TP + FN) + (TN + FN)(TN + FP)}{(TP + TN + FP + FN)^2}$$

4.2 Federated learning

Several edge nodes must collaborate to develop a unified learning model through local training, which necessitates the use of distributed learning techniques. Federated Learning (FL), unlike traditional centralized learning frameworks, is a decentralized approach that preserves data privacy and confidentiality by ensuring that each local model is trained exclusively on data residing at the corresponding edge node. In this paradigm, communication between the edge nodes and the cloud server is limited to the exchange of updated model parameters rather than raw data, thereby significantly reducing transmission latency and communication overhead. By retaining the training data at the edge nodes and transmitting only the locally optimized parameters to a global aggregation server, federated learning (FL) enables $\mathcal{K}$ edge nodes to collaboratively construct a shared learning model, as illustrated in Fig. 5. In this framework, each edge node independently trains a local IDS model using its private dataset and periodically sends the updated parameters or gradients to the aggregation server localized at the cloud computing.

Using the federated averaging algorithm (FedAvg), the global model is the (usually data-size-weighted) average of the local models:

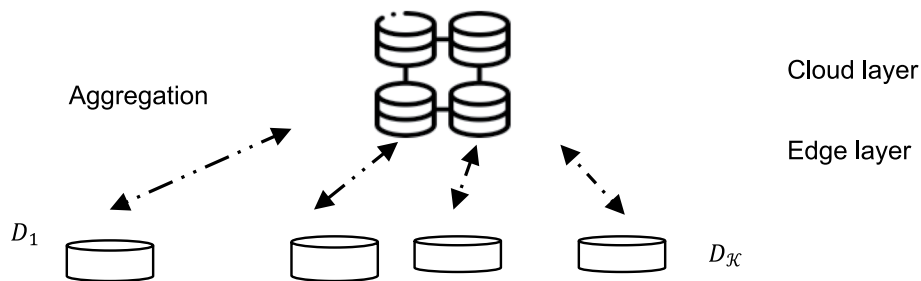


Fig. 5 Overview of the proposed FL for IDS model

$$W^{(t+1)} = \sum_{k \in \mathcal{K}_t} \frac{n_k}{j \in \mathcal{K}_t} W_k^{(t+1)} \quad (36)$$

where, $W_k^{(t+1)}$ is the local model at edge k after local training in round t , n_k is the number of local training samples at edge k , $\mathcal{K}_t$ is the set of participating edges in round t

The FL model supports collaborative training of DL models for edge networks. FL model further improves generalization by exposing the global model to heterogeneous data distributions across multiple data centers. The learning procedures are summarized as follows:

- *Global model initialization:* A global IDS model is initialized by the cloud server and delivered to the clients at the beginning of each communication round.
- *Local training:* Each edge device uses its private data to train the received global model and produce a local model.
- *Client aggregation:* The local models from the edge nodes are aggregated by the cloud sever to update the global model parameters.

The federated optimization process employs stochastic gradient descent (SGD), with a single batch-gradient computation performed during each communication round. In this study, we employ federated averaging (FedAvg), which is built upon federated stochastic gradient descent (FedSGD) to improve accuracy and reduce communication costs. The cloud server computes the new global model parameters by averaging the local models from all participating edge nodes and uses the result for the next iteration. This process is repeated until convergence is achieved or a predefined stopping condition is met.

Our ultimate goal is to minimize the loss function $\mathcal{L}_i(w; x_i, y_i)$, which represents the loss over data sample (x_i, y_i) given model parameters $w \in \mathbb{R}^d$. To this aim, we focus on non-convex neural network objective, denoted $\mathbb{h}$. Then, we consider the finite-sum scenario for compositional optimization of a sytem with n point:

$$\min_{w \in \mathbb{R}^d} c, \text{ where } \mathbb{h}(w) \stackrel{\text{def}}{=} \frac{1}{n} \sum_{i=1}^n \mathbb{h}_i(w)$$

Now, consider a system with $\mathcal{K}$ edge nodes over which the data is partitioned, with S_k on edge node k .

$$\mathbb{h}(w) = \sum_{k=1}^{\mathcal{K}} \frac{n_k}{n} \mathbb{H}_k(w), \text{ where } \mathbb{H}_k(w) = \frac{1}{n_k} \sum_{i \in S_k} \mathbb{h}_i(w)$$

The dataset is partitioned across the nodes such that each edge node k holds a local dataset of size n_k , with $n_k = \mathcal{D}_k = \frac{|\mathcal{D}|}{\mathcal{K}}$. Each model is trained locally using its corresponding dataset $\mathcal{D}_k$. For a supervised machine learning problem, the local loss function is typically defined as:

$$\mathbb{h}_i(w) = \mathcal{L}_i(w; x_i, y_i)$$

where (x_i, y_i) denotes the input-label pair of the i -th data sample.

In FedSGD model, the objective is to find the optimal model parameters w^* that minimize the global loss function:

$$w^* = \arg \max_w \frac{1}{\mathcal{K}} \sum_{t=1}^{\mathcal{K}} (w; x_t, y_t) \quad (37)$$

During a training round, a predetermined number of clients $\mathcal{M}$ participate in updating the global model. The final model is updated using the aggregated gradients computed by the participating edge nodes [44]:

$$w_t \leftarrow w_{t-1} - \gamma \frac{1}{\mathcal{M}} \sum_{t=1}^{\mathcal{M}} \nabla \mathcal{L}(w_{t-1}; x_t, y_t)$$

where γ is the learning rate.

Let g_k denote the local gradient computed at edge node, defined as:

$$g_k = \nabla \mathbb{H}_k(w_t)$$

The global gradient of the objective function can then be expressed as:

$$\sum_{k=1}^{\mathcal{K}} \frac{n_k}{n} g_k = \nabla \tilde{\mathcal{L}}(w_t)$$

Accordingly, the global model update rule in FedSGD is given by:

$$w_{t+1} \leftarrow w_t - \gamma \sum_{k=1}^{\mathcal{K}} \frac{n_k}{n} g_k \quad (38)$$

FedAvg builds upon FedSGD by aggregating multiple local updates before communication, thereby reducing communication overhead while preserving the security guarantees of federated learning. FedAvg enables multiple local updates per client per round; therefore, the model considers the edge participation fraction $\mathcal{Z}$, the number of local epochs E , and the mini-batch size B . In the first step, the cloud server randomly selects a subset of edge nodes $|S_t| = \mathcal{Z} \times K$, where $\mathcal{M} = \max(\mathcal{Z} \times K, 1)$, to participate in the current training round. At each round t , the edge nodes transmit their trained local models w_{t+1}^k to the central cloud server for aggregation to create a new global mode w_{t+1} [44].

$$\forall k, w_{t+1}^k \leftarrow w_t - \gamma g_k \text{ and } w_{t+1} \leftarrow \sum_{k=1}^{\mathcal{K}} \frac{n_k}{n} w_{t+1}^k$$

Basically, we use logits for the target output classes to optimize the cross-entropy loss function τ_{CE} (Nanayakkara et al. [45]).

$$\tau_{\text{CE}}(y, \mathcal{Y}(x)) = -\log \frac{\exp(\mathcal{Y}_y(x))}{\sum_{y' \in \mathbb{C}} \exp(\mathcal{Y}_{y'}(x))} \quad (39)$$

where $\mathbb{C}$ is the label space $\mathcal{Y}_{y'}(x)$ is the output logits for class y'

For a FedAvg model with parameters w , the training dataset is partitioned into multiple mini-batches $B = \{1, \dots, |B|\}$, where $j \in B$ indexes a given batch. The loss function for batch is defined as:

$$\mathcal{L}(w, j) = \tau_{CE}(y, \mathcal{Y}(x))$$

where τ_{CE} denotes the cross-entropy loss, y represents the ground-truth labels, $\mathcal{Y}$ is the model prediction for input x .

After completing a full learning round, all participating nodes receive the global model aggregated using the proposed FedAvg algorithm, as illustrated in the following pseudo-code.

Input: local dataset $\mathcal{D} = \{\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_K\}$; Initial global model w^0
 Edge computing $Ed_i = \{1, 2, \dots, K\}$; Number of local epochs E
 Size of batch $|B|$; batch $\mathcal{B}$

Output: Global model w

```

1 : Procedure cloud computing
2 :   Initialize  $w_0$ 
3 :   For each round  $t = 1, 2, \dots, T$  do
4 :      $S_t = \text{random set of } \max(\mathcal{Z}, K, 1) \text{ of edge nodes}$ 
5 :     For every edge  $k \in S_t$  in parallel do
6 :        $w_{t+1}^k \leftarrow \text{EdgeUpdate}(k, w_t)$ 
7 :        $w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$  combine client models
8 :     End for
9 :   End for
10 : Procedure edge computing ( $k, w$ )
11 :   For each local epoch  $i \in (1, \dots, E)$  do
12 :      $\mathcal{B} \leftarrow \text{split } \mathcal{D} \text{ into batches of size } |B|$ 
13 :     For  $\mathcal{B} \in \mathcal{B}$  do
14 :        $w \leftarrow w - \gamma \nabla \ln(w, \mathcal{B})$ 
15 :     End for
16 :   End for
17 : Return  $w$  to cloud computing
  
```

Algorithm 4 FedAVG based IDS

The presence of non-independent and identically distributed (non-IID) data introduces several challenges, including imbalanced class distributions, heterogeneous local model updates, and increased communication overhead, all of which negatively affect the convergence speed and overall performance of FL models. To this end, FedAvg algorithm is widely employed to mitigate these challenges and has proven effective in decentralized learning environments characterized by non-IID data, which commonly arise due to geographical and temporal heterogeneity. To simulate a realistic FL environment, dataset is partitioned across edge nodes with skewed class distributions, such that certain attack types are over-represented on specific edge nodes while being under-represented or entirely absent on others. This data heterogeneity realistically reflects real-world data center conditions, where traffic patterns and attack characteristics vary across different network locations and over time.

4.3 Computational complexity and communication overhead analysis

For TL model, the computational complexity is primarily associated with the pretraining phase (offline), which is typically performed once on large-scale datasets, with a computational complexity defined as:

$$\mathcal{O}(E_p \cdot N_p \cdot C)$$

where E_p is the number of pretraining epochs,

N_p is the dataset size, C is per-sample model cost.

This cost is amortized and does not impact deployment-time feasibility. Moreover, fine-tuning only the last k layers reduces complexity to

$$\mathcal{O}(E_f \cdot N_t \cdot C_k), C_k \ll C$$

where N_t is the task-specific dataset size.

This makes TL computationally efficient for data center workloads. Regarding communication overhead, TL incurs a one-time transmission cost, as the pretrained model is sent only once from the central server to the target edge node. The corresponding communication complexity is:

$$\mathcal{O}(|\theta|)$$

where $|\theta|$ is the model parameter size.

In contrast, for FL model, the computational complexity is primarily dominated by local training at each edge. The local computation cost per edge is defined as:

$$\mathcal{O}(E_l \cdot N_c \cdot C)$$

where E_l is the number of local epochs and N_c the edge dataset size.

In FedAvg, the server computes a weighted average of edge model updates, resulting in linear-time aggregation with respect to the total number of model parameters.

$$\mathcal{O}(K \cdot |\theta|)$$

here K is the number of participating edge nodes.

Consequently, the server-side aggregation complexity scales linearly with the number of participating edge nodes, enabling efficient parallelization across data center resources. In this distributed training paradigm, the communication overhead incurred by model uploads and downloads is given by: $\mathcal{O}(K \cdot |\theta|)$. In each round, every participating edge node uploads a local model update and downloads the aggregated global model. Consequently, the total overhead across R rounds is:

$$\mathcal{O}(R \cdot K \cdot |\theta|)$$

Overall, the analysis demonstrates that both TL and FL models are computationally feasible and communication-efficient, thereby confirming their suitability for deployment in real-world data center environments. TL enables efficient deployment with minimal computational overhead, whereas FL incurs higher communication costs in exchange for improved scalability and enhanced data privacy. This trade-off highlights the practicality of hybrid TL–FL frameworks in contemporary distributed data centers.

4.4 Explainable artificial intelligence

As the concepts of FL and TL are used as black boxes, these models cannot provide any information about the reasons behind IDS decisions. As a result, it is critical to use explainable artificial intelligence (XAI) methods to increase AI transparency and provide deeper insight into the decisions made by IDS models. Figure 6 presents an overview of how XAI is used to interpret the predictions of the proposed IDS model.

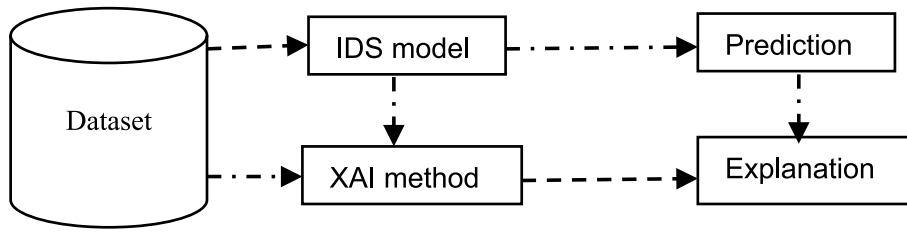


Fig. 6 Overview of how to use XAI to interpret the predictions of IDS model

In this study, we investigate the problem of revealing the functionality of a black-box IDS model by using XAI techniques. Explanation methods can be broadly classified into global model explainability, local (outcome-based) explainability, and black-box inspection techniques. Many high-performing XAI techniques based on feature relationships have been developed, including Contextual Importance and Utility (CIU), Shapley Additive Explanations (SHAP), Model-Agnostic Post-hoc Local Explanations (MAPLE), global surrogate models, Accumulated Local Effects (ALE), Local Interpretable Model-Agnostic Explanations (LIME), and Local Rule-based Explanations (LORE) [46].

On the one hand, we use a cooperative game theory-based method known as SHAP values to increase transparency. This method explains how dataset features affect the output of the IDS model in edge data centers.

We formulate a feature selection procedure based on a cooperative game, where representative feature subsets are modeled as a payoff function $\mathbb{Q}$.

$$\mathbb{Q} = (\mathcal{N}, \vartheta),$$

where $\mathcal{N} = \{1, 2, \dots, n\}$ is the set of features used to build the IDS model, $\mathcal{S} \subseteq \mathcal{N}$ is any coalition (subset) of features, $\vartheta: 2^{\mathcal{N}} \rightarrow \mathbb{R}$ denote the payoff (importance) assigned to a feature.

Formally, $\vartheta(\mathcal{S})$ represents the contribution, utility, or predictive power of the IDS model when only the features in coalition $\mathcal{S} \subseteq \mathcal{N}$ are used. In this case, this function satisfies: $\vartheta(\emptyset) = 0$ for an empty set $\emptyset$.

Shapley values explain a model by expressing the contribution of individual feature $i \in \mathcal{N}$ to the predictions of the model as follows [47]:

$$\varphi_i(\vartheta) = \sum_{\mathcal{S} \subseteq \mathcal{N} \setminus \{i\}} \frac{|\mathcal{S}|! (|\mathcal{N}| - |\mathcal{S}| - 1)!}{|\mathcal{N}|!} [\vartheta(\mathcal{S} \cup \{i\}) - \vartheta(\mathcal{S})] \quad (40)$$

Shapley value $\varphi(\cdot)$ is the unique payoff allocation in a cooperative game theory that satisfies four fundamental axioms: efficiency, symmetry (equal payoff for equally contributing features), dummy player (zero payoff for useless features), and additivity.

The total contribution of all features is fully distributed among individual features:

$$\sum_{i \in \mathcal{N}} \varphi_i(\vartheta) = \vartheta(\mathcal{N})$$

If two features contribute equally to all coalitions, they receive equal payoffs. Formally, for any $i, j \in \mathcal{N}$,

$$\text{If } \vartheta(\mathcal{S} \cup \{i\}) = \vartheta(\mathcal{S} \cup \{j\}), \forall \mathcal{S} \subseteq \mathcal{N} \setminus \{i, j\}, \text{ then } \varphi_i(\vartheta) = \varphi_j(\vartheta)$$

If a feature does not add any marginal contribution to any coalition, it receives zero payoff. Formally, for some $i \in \mathcal{N}$

If $\vartheta(\mathcal{S} \cup \{i\}) = \vartheta(\mathcal{S})$, $\forall \mathcal{S} \subseteq \mathcal{N} \setminus \{i\}$, then $\varphi_i(\vartheta) = 0$

For any two coalitional games $\mathbb{Q}_1 = (\mathcal{N}, \vartheta_1)$ and $\mathbb{Q}_2 = (\mathcal{N}, \vartheta_2)$,

If $(\vartheta_1 + \vartheta_2)(\mathcal{S}) = \vartheta_1(\mathcal{S}) + \vartheta_2(\mathcal{S})$, $\forall \mathcal{S} \subseteq \mathcal{N}$, then $\varphi_i(\vartheta_1 + \vartheta_2) = \varphi_i(\vartheta_1) + \varphi_i(\vartheta_2)$, $\forall i \in \mathcal{N}$

Unfortunately, Shapley value is frequently hindered by its computational cost, since it must evaluate all $2^{\mathcal{N}}$ possible feature subsets. The most effective way to overcome this limitation is to use the average of these marginal contributions to provide an estimated Shapley value for each feature. To this aim, we propose a polynomial-time approximation of Shapley values to assess the significance of features when using deep neural network-based IDS models. For all possible orders of the features in $\mathcal{N}$, we define the permutation $\mathcal{P}$ that assigns to each position k the corresponding feature, denoted by $\mathcal{P}(k)$.

Given a permutation $\mathcal{P}$, we denote $\text{Pred}^i(\mathcal{P})$ as the set of predecessors of feature i in the order $\mathcal{P}$; that is, $\text{Pred}^i(\mathcal{P}) = \{j \in \mathcal{N} : \mathcal{P}(j) < \mathcal{P}(i)\}$. We also denote $\mathbb{G}(\mathcal{N})$ as the set of all possible permutations of the feature set $\mathcal{N}$. In this case, $|\mathcal{N}|$ is the cardinality of all features. The permutation $\mathcal{P}$ is defined as a bijective mapping:

$$\mathcal{P} : \mathcal{N} \rightarrow \{1, \dots, n\}, \text{ where } n = |\mathcal{N}|$$

Accordingly, $\mathcal{P}(i)$ denotes the position (rank) of feature i in the ordering $\mathcal{P}$. Based on this formulation, we employ SHAP explanations to quantify the contribution of each feature to the predictions of the CNN-based model. In this study, we adopt a Shapley value approximation based on random sampling, following the approach proposed by Castro et al. [48] and further adapted for deep learning models by Ancona et al. [49].

$$\varphi_i(\vartheta) = \sum_{\mathcal{P} \in \mathbb{G}(\mathcal{N})} \frac{1}{|\mathbb{G}(\mathcal{N})|} (\vartheta(\text{Pred}^i(\mathcal{P}) \cup i) - \vartheta(\text{Pred}^i(\mathcal{P}))), \quad i = 1, \dots, n \quad (41)$$

On the other hand, we interpret the model by randomly altering each feature column's values, one column at a time, using Permutation Feature Importance (PFI). The interpretability of CNN models can be improved by using the PFI approach to complement the SHAP method.

Let $\mathcal{N}$ be the set of features ($n = |\mathcal{N}|$) and let $\mathcal{S} \subseteq \mathcal{N}$ with its complement $\bar{\mathcal{S}} = \mathcal{N} \setminus \mathcal{S}$. Given observations $(x_1, y_1), \dots, (x_n, y_n)$, we calculate the PFI weight for each feature when using the CNN model, denoted $\mathbb{F}_s$. The Euclidean norm $\|\cdot\|$ is used to define the PFI weights as follows [50, 51]:

$$\bar{\mathcal{U}}^{\mathcal{S}_i} = \frac{1}{n(n-1)} \sum_{k=1}^n \sum_{j \neq k} \left\| \mathbb{F}_s \left(x_k^{\bar{\mathcal{S}}_i}, x_j^{\mathcal{S}_i} \right) - y_k \right\| - \frac{1}{\mathcal{N}} \sum_{k=1}^{\mathcal{N}} \left\| \mathbb{F}_s(x_k) - y_k \right\| \quad (42)$$

In this study, we advocate the use of PFI and SHAP as complementary interpretability approaches, as they provide different yet synergistic perspectives on feature importance. PFI offers a lightweight and model-agnostic overview of global feature relevance, making it suitable for rapid assessments under limited computational resources. However, since our work focuses on deep learning-based intrusion detection systems (IDS), which exhibit complex non-linear behaviors and feature interactions, we primarily base

our analysis on SHAP. Owing to its solid theoretical foundation and its ability to capture higher-order feature interactions, SHAP provides more faithful and fine-grained explanations of model decisions.

4.5 Performance evaluation

The preprocessing phase includes data cleaning, scaling, and feature selection, followed by the training, validation, and testing of the IDS model, as shown in Fig. 7.

Note that IDS incidents generally fall into four categories: false positive (FP), false negative (FN), true positive (TP), and true negative (TN). In an FP case, the model incorrectly predicts a negative class as positive. In a TN case, the model correctly predicts the class label as negative. In a true positive (TP) case, the IDS model correctly predicts a positive class label. Based on these definitions, the four statistical performance measures can be computed as follows using the aforementioned terms [52].

$$\text{Accuracy} = \text{Ac} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

$$\text{Precision} = \text{Pr} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \text{Re} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{F1 - Score} = \text{F1} = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}}$$

Additionally, we use two advanced metrics for binary classification, namely Matthews's Correlation Coefficient (MCC) and the Geometric Mean (G-mean). These metrics can be expressed as follows:

$$\text{MCC} = \frac{\text{TP} \times \text{TN} - \text{FP} \times \text{FN}}{\sqrt{(\text{TP} + \text{FP}) \times (\text{TP} + \text{FN}) \times (\text{TN} + \text{FP}) \times (\text{TN} + \text{FN})}}$$

$$\text{G - Mean} = \sqrt{\frac{\text{TN}}{\text{TN} + \text{FP}} \times \frac{\text{TP}}{\text{TP} + \text{FN}}}$$

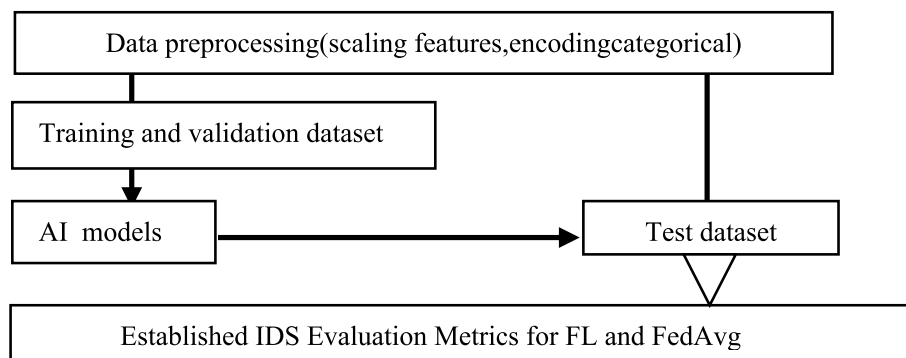


Fig. 7 Proposed AI-based IDS framework

We also employ the confusion matrix as an evaluation metric, as it incorporates all four outcomes, TP, FP, FN, and TN, providing a comprehensive assessment of the model's performance, as shown in Table 1.

4.6 Hyperparameter setting

Hyperparameter tuning is the process of selecting the optimal set of values and algorithms for a CNN model to enhance its performance. More importantly, hyperparameter tuning helps optimize a deep learning model by controlling the bias–variance trade-off, improving its ability to generalize. Adjusting parameters slightly to achieve even greater performance is known as fine-tuning. Note that fine-tuning during transfer learning involves releasing and unfreezing some or all of the pre-trained model's layers to allow it to better adapt to the current task. This approach is used to determine the optimal hyperparameters for a CNN model, such as batch size, number of hidden layers, number of epochs, activation function, optimizer, learning rate, and filter sizes. Moreover, Tree-structured Parzen Estimator (TPE) splits the data into good and bad groups and then uses the density ratio of these groups as an acquisition function (AF). After separating the data, the TPE algorithm uses this density ratio as the acquisition function to guide the sampling process. In this study, we use TPE because it is fast, simple, and non-parametric, with a runtime complexity of $O(n \log n)$.

For transfer learning, all pre-trained CNN models are fine-tuned under identical experimental conditions to ensure a fair comparison. ImageNet pre-trained weights are adopted, and the models are trained for 10 epochs using the Adam optimizer with a learning rate of 0.0001 and a batch size of 32. Categorical cross-entropy is employed as the loss function, with ReLU activation in hidden layers and Softmax in the output layer. The input image size is set to 256×256 , and dropout with a rate of 0.5 is applied to reduce overfitting.

To ensure reproducibility, the federated learning framework is configured with 10 distributed clients, each holding a local partition of the UNSW-NB15 dataset. These clients update a shared global model locally several times to improve communication efficiency. For the FedAvg model, the number of epochs is set to 600 for each global model update during synchronization. Training is performed using FedAvg model, also known as Local SGD, over 5 communication rounds ($R=5$), where all edge nodes participate in each round. During every round, each edge node conducts 2 local training epochs ($E=2$) on its private data using a mini-batch size of 32 and the Adam optimizer with a learning rate of 0.0001. After local training, edge's model updates are transmitted to a central cloud computing and aggregated using a data-size weighted average, following the standard FedAvg strategy. The resulting global model is then redistributed to all edge nodes for the next communication round until convergence is achieved.

For classical machine learning baselines, the SVM classifier is configured with an RBF kernel, a regularization parameter $\gamma = 1.0$, a kernel coefficient $\gamma = \text{scale}$, and balanced class

Table 1 Confusion matrix demonstration

		Predicted value	
		Positive	Negative
Actual value	Positive	True positive (TP)	False negative (FN)
	Negative	False positive (FP)	True negative (TN)

weights, while the KNN model uses $k = 5$ nearest neighbors with Euclidean distance and uniform weighting.

All hyperparameter settings are carefully selected and consistently applied to ensure a fair, unbiased, and reproducible comparison across both deep learning-based IDS models and traditional machine learning classifiers, thereby enabling a reliable and objective evaluation.

5 Simulation result

The aim of this study is to build a robust IDS system that leverages an advanced deep learning architecture for classifying attacks and monitoring network flow in large-scale AI data centers. To conduct the experiments, we use the UNSW-NB15 dataset [53] for training and testing the proposed IDS models. In the dynamic landscape of edge computing, we perform both binary and multiclass classification of attacks using benign behavior as well as contemporary synthesized attack activities. In its clean format, this dataset contains 42 features and includes nine types of cyberattacks: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. The dataset was partitioned into training and testing sets, with 70% allocated for training and 30% reserved for testing.

5.1 Transfer learning

To build efficient IDS models for edge data centers, convolutional operations are performed on RGB images. Since CNN classifiers are designed for image classification, we convert the dataset into image matrices using the OpenCV library to create RGB representations. This representation facilitates the learning of higher-level abstractions and complex inter-feature relationships that are less effectively captured by traditional tabular models. Moreover, similar tabular-to-image encoding strategies have been successfully adopted in prior intrusion detection and traffic classification studies, demonstrating improved performance over conventional deep learning approaches. After this transformation, we label the generated images based on the attack pattern of each data chunk. This process is repeated until all labels in the dataset are correctly transformed into images by mapping features to pixel positions. Finally, the UNSW-NB15 dataset is converted into RGB image blocks, which improves the learning capability of the proposed IDS, as illustrated in Fig. 8.

In binary classification, we investigate the performance of six state-of-the-art deep neural network CNN architectures, including Base CNN (BCNN), VGG16, DenseNet121 (DN121), ResNet-50 (RN50), EfficientNetB0 (ENB0), InceptionV3 (IV3), and MobileNetV3-Small (MNVS). We now consider the results obtained with these various pre-trained CNN models after 10 epochs of training on the UNSW-NB15 dataset, as shown in Table 2.

According to the experimental simulation results, we found that ResNet50, InceptionV3, and DenseNet121 outperform the other pre-trained CNN models as well as the traditional CNN model (CNN Only). More specifically, the ResNet50 model achieves the best performance among the six state-of-the-art CNN architectures. Therefore, these top-performing models are selected to develop the ensemble IDS model. To achieve this, we first calculate the Kappa coefficient and the associated weight for each model, as shown in Table 3.

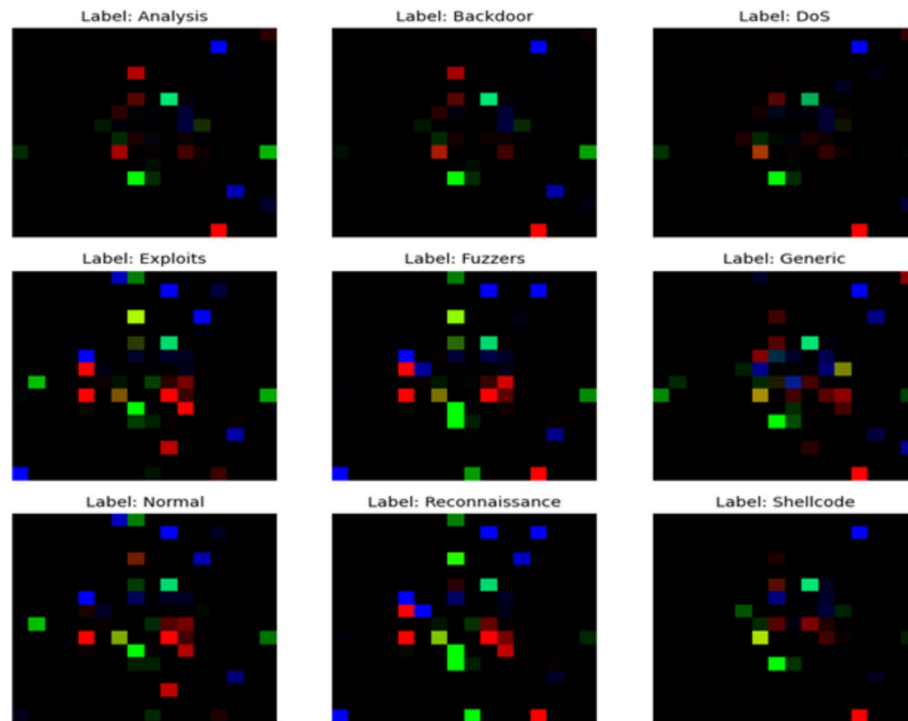


Fig. 8 Images obtained from the UNSW-NB15

Table 2 Performance comparison of the pre-trained CNN models

Models	Ac	Pr	Re	F1	MCC	G-mean
BCNN	0.9986	0.9986	0.9986	0.9986	0.9972	0.9985
RN50	0.9998	0.9998	0.9998	0.9998	0.9996	0.9998
VGG16	0.9986	0.9986	0.9986	0.9986	0.9972	0.9986
MNVS	0.7936	0.8061	0.7936	0.7938	0.5993	0.8002
ENB0	0.8053	0.8638	0.8053	0.8017	0.6712	0.8226
IV3	0.9995	0.9995	0.9995	0.9995	0.9989	0.9994
DN121	0.9993	0.9993	0.9993	0.9993	0.9987	0.9993

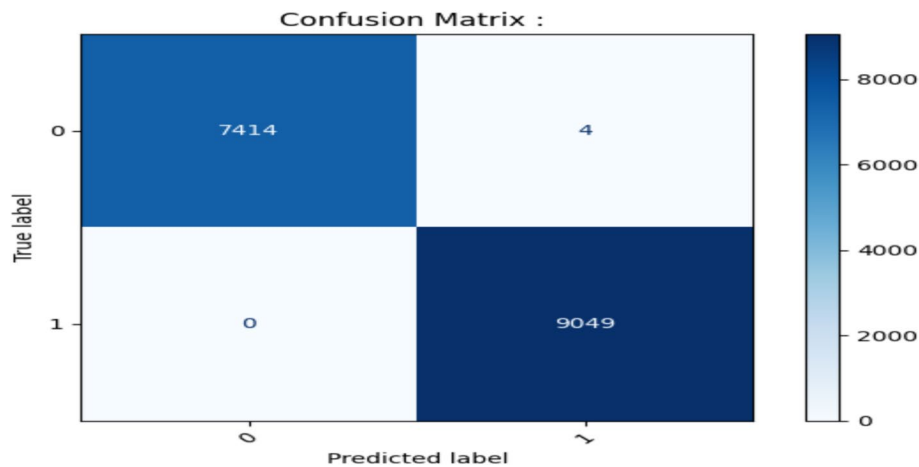
Table 3 Kappa coefficients and their associated weights

	ResNet50	InceptionV3	DenseNet121
Kappa coefficient	0.9985	0.9995	0.9994
Weights (w_i)	7.2137	8.3127	8.0894

The effectiveness of the weighted modeling strategy is reflected in the strong correspondence between the Kappa coefficients and the assigned weights across the evaluated architectures. Accordingly, InceptionV3 achieves the highest Kappa coefficient (0.9995) and is consequently assigned the largest weight (8.3127), making it the most influential contributor to the ensemble. DenseNet121 follows closely, with a Kappa coefficient of 0.9994 and a slightly lower assigned weight (8.0894). In contrast, ResNet50, despite maintaining a high level of agreement (0.9985), receives the smallest weight (7.2137). This performance-driven weighting strategy ensures that models with superior predictive reliability exert a stronger influence on the final ensemble decision, thereby enhancing robustness and stability while mitigating the impact of comparatively weaker predictors.

Table 4 Performance comparison of base CNN and TL model

Model	Accuracy	Precision	Recall	F1-score
Base CNN	0.9986	0.9986	0.9986	0.9986
TL model	0.9998	0.9996	1.0000	0.9998

**Fig. 9** Confusion matrix for TL I in a binary classification

Therefore, the simulation results validate the effectiveness of the proposed TL-based ensemble model, which combines ResNet50, InceptionV3, and DenseNet121 using a Kappa-based weighting scheme. The results demonstrate that the proposed model consistently outperforms the individual base CNN models, as summarized in Table 4.

The transfer learning model consistently outperforms the base CNN across all evaluation metrics, achieving an accuracy of 0.9998, precision of 0.9996, perfect recall of 1.0000, and the highest F1-score of 0.9998, compared to the base CNN model, which attains 0.9986 across all metrics. Although the numerical improvements are marginal due to the already near-optimal baseline performance, these results clearly demonstrate the effectiveness of leveraging pre-trained representations to further enhance model generalization and robustness.

In general, near-perfect performance on benchmark datasets such as UNSW-NB15 does not necessarily indicate overfitting, particularly when the evaluation protocol and model design explicitly incorporate mechanisms to promote generalization. In our framework, overfitting is mitigated through several complementary strategies, including regularization, early stopping, and ensemble learning, which reduce reliance on dataset-specific noise. Moreover, feature selection is applied in a data-driven manner to eliminate redundant and irrelevant features rather than to tailor the model to the test set.

Importantly, we evaluate the efficacy of the classification model by comparing the actual and predicted outcomes using the confusion matrix, which helps in identifying sources of confusion and error. This two-by-two matrix combines precision and recall through the harmonic mean to provide a concise summary of the performance of the proposed hybrid model. It also demonstrates how effectively the proposed TL model detects cyber-attack types in cloud-edge data centers. As illustrated in Fig. 9, the binary classification clearly distinguishes between benign and attack classes when applying the TL technique to the publicly available UNSW-NB15 dataset.

Table 5 Performance of pre-trained models and basic CNN model

Label	CNN	DN121	IV3	ENB0	MNVS	RN50	VGG16	TL
Analysis	5.38	59.54	59.54	0.00	0.00	55.70	8.40	58.96
Backdoor	54.87	46.15	51.28	0.00	0.00	51.30	27.35	46.22
DoS	80.28	91.22	92.49	2.50	3.90	92.00	89.17	92.87
Exploits	87.03	90.86	90.24	0.10	11.10	91.40	88.02	92.53
Fuzzers	89.03	90.26	89.60	0.00	16.20	91.40	89.33	88.94
Generic	98.50	99.57	99.70	98.50	29.20	99.60	99.73	99.70
Normal	99.53	99.85	99.95	65.60	66.80	99.90	99.85	99.97
Reconnaissance	82.71	95.71	96.82	0.00	0.00	96.50	95.97	96.94
Shellcode	89.33	100	97.33	23.20	12.60	78.90	100	100
Worms	25.00	71.43	100.00	0.00	0.00	53.80	71.43	100

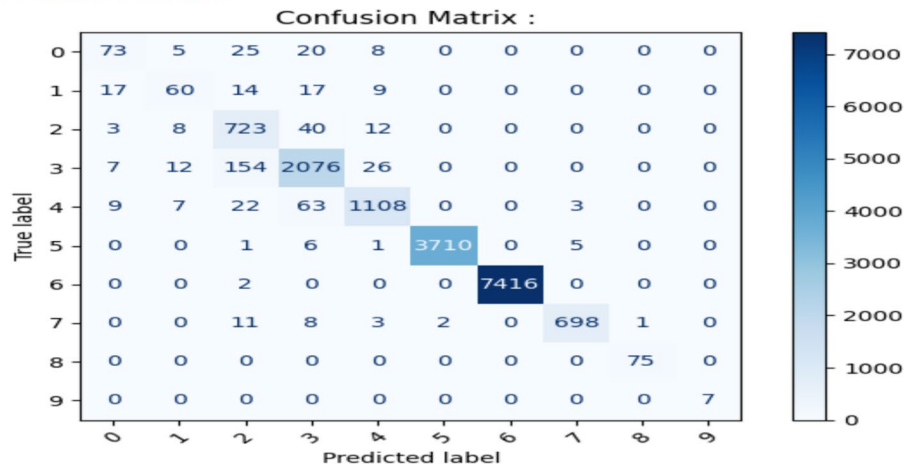
✓ Evaluation Metrics:

Accuracy : 0.9684

Precision : 0.9695

Recall : 0.9684

F1-Score : 0.9684

**Fig. 10** Confusion matrix for the ResNet-50 network

It provides a clear, concise, and reliable score for evaluating binary classifications compared with other traditional performance metrics. From the confusion matrix, it is evident that the proposed FL model demonstrates a high performance level in terms of correct and incorrect classifications (TN and TP).

On the other hand, we extend IDS to multiclass classification to further identify specific attack types. The objective of this part of the study is to evaluate the performance of the pre-trained CNN models and compare them with the proposed federated learning (FL) approach in terms of accuracy, as shown in Table 5.

Simulation results prove that the proposed TL model outperforms other CNN models in terms of average accuracy, achieving 87.613% and 81.05% for TL and ResNet50, respectively. In this study, we focus on the confusion matrix that uses network flows to study and classify various types of cyberattacks. For simplicity, we denote the attack types, fuzzers, analysis, backdoors, DoS, exploits, generic, reconnaissance, shellcode, and worms, as sequential numbers from 0 to 9. In this context, we have studied and analysed the performance of the three selected pre-trained models. First, the following simulation results summarise the performance of the ResNet50 model by comparing its predicted labels to the true labels, as shown in Fig. 10.

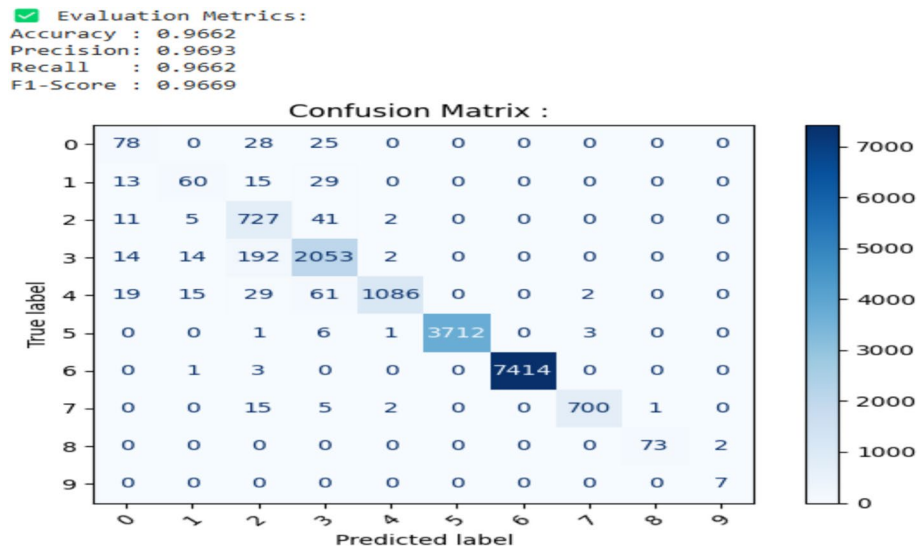


Fig. 11 Confusion matrix for the inceptionV3 network

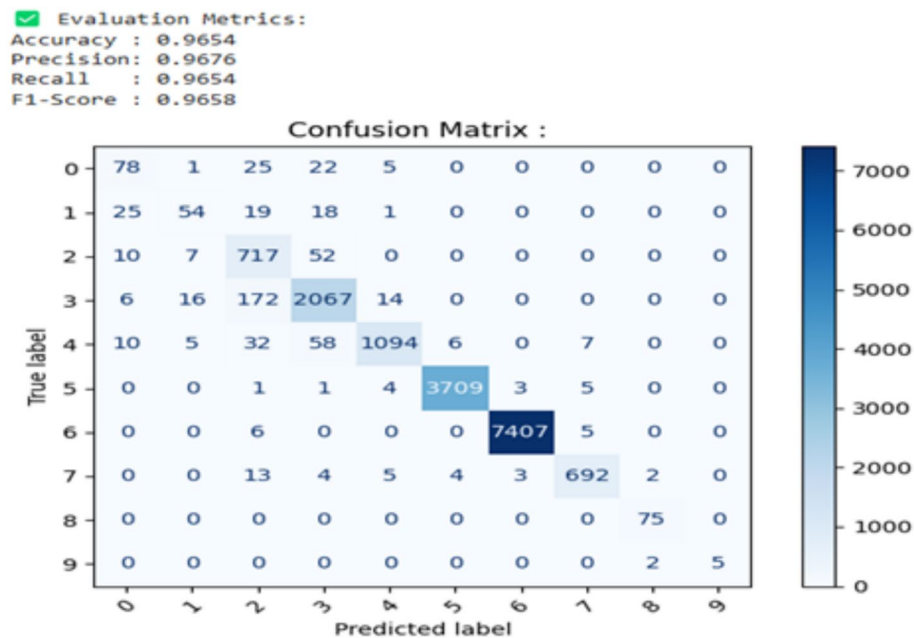


Fig. 12 Confusion matrix for the DenseNet121 network

Along the same lines, to provide a comprehensive overview of the performance of the proposed algorithm, Fig. 11 illustrates the confusion matrix of the InceptionV3 model.

Lastly, the classification results of the proposed DenseNet121 model are illustrated in Fig. 12 in the form of a confusion matrix.

This work presents a unified framework designed to leverage the complementary strengths of the previously introduced pre-trained models. Accordingly, Fig. 13 illustrates the multi-class classification performance of the proposed weighted TL-based ensemble model.

For comparison purposes, the performance of the baseline CNN model is evaluated using its corresponding multi-class confusion matrix, as illustrated in Fig. 14.

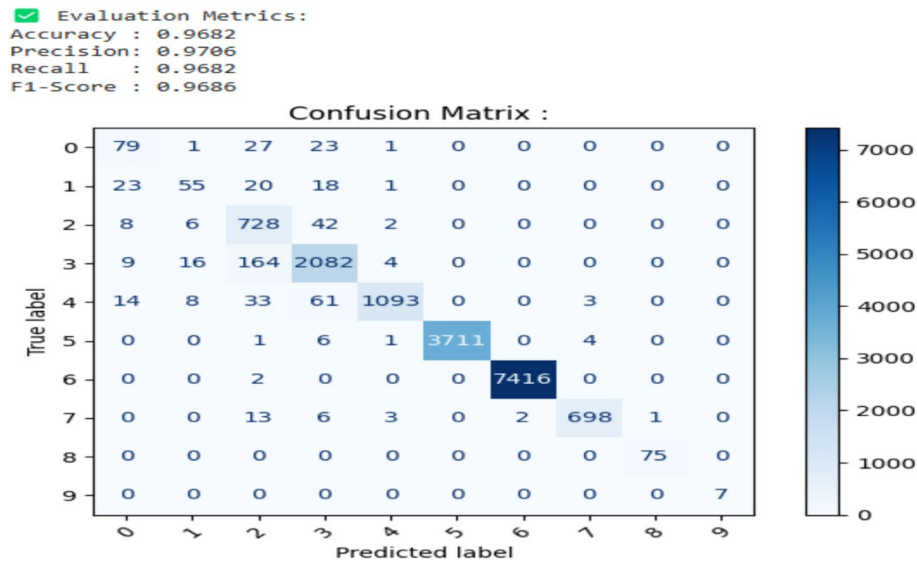


Fig. 13 Confusion matrix for the proposed TL model

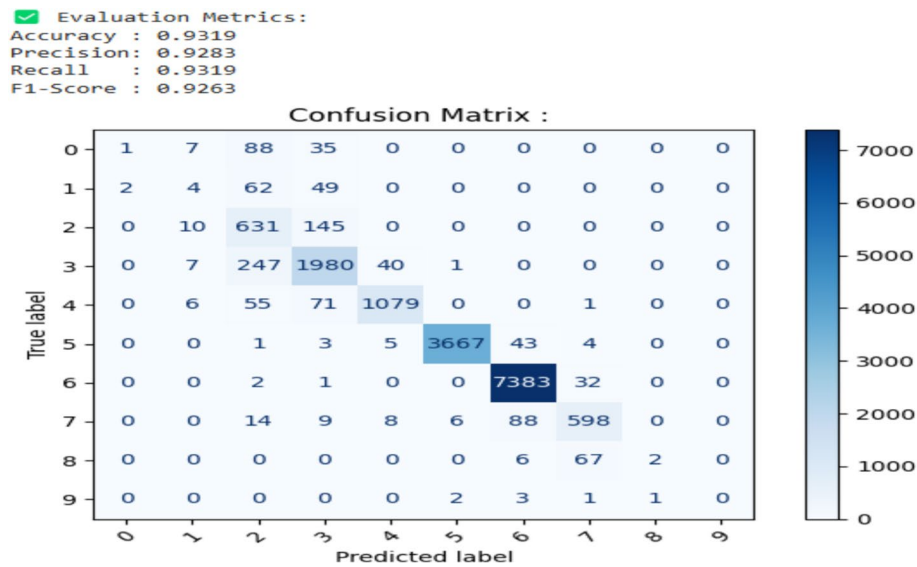


Fig. 14 Confusion matrix for the basic CNN model

Based on the above classification results, the confusion matrix visually highlights the superior performance of the proposed approach, clearly demonstrating the enhanced effectiveness of network intrusion detection achieved by the weighted CNN model. Notably, the F1-score serves as a unified performance metric that combines precision and recall into a single measure, providing a balanced assessment of the model's classification capability.

5.2 Federated learning

The proposed decentralized technique, based on federated learning, is employed in this study to train an intrusion detection system (IDS) model across multiple edge servers holding heterogeneous datasets, without sharing raw data among them. In practice, we create ten distinct edge nodes by splitting the original dataset into ten portions. First,

we identify the most important, impactful, and non-redundant features in the dataset to determine an optimal feature set for constructing efficient IDS models.

5.2.1 Feature selection

The objective is to enhance the CNN's classification performance and reduce computation time by eliminating features that are irrelevant or only partially relevant, ensuring that their removal does not negatively impact model performance. In this context, PSO, GO, and WOA methods are employed to select the most significant features for the proposed IDS model in data centers. Furthermore, this paper introduces a novel strategy that integrates these methods using both intersection and union operations. With respect to feature selection, we first highlight its critical importance in CNN-based IDS models, placing particular emphasis on the PSO, GOA, and WOA techniques, as illustrated in Fig. 15.

The feature importance analysis using PSO, GOA, and WOA demonstrates a strong consensus on the most influential network traffic attributes for intrusion detection. Core features such as service, ackdat, ct_dst_src_ltm, ct_src_ltm, ct_srv_dst, ct_dst_sport_ltm, and synack consistently receive high importance scores across all three optimization methods, indicating their critical role in capturing abnormal communication patterns and attack behaviors in data centers. Conversely, features including state, rate, dload, sload, proto, ct_dst_sport_ltm, and ct_state_ttl show comparatively lower

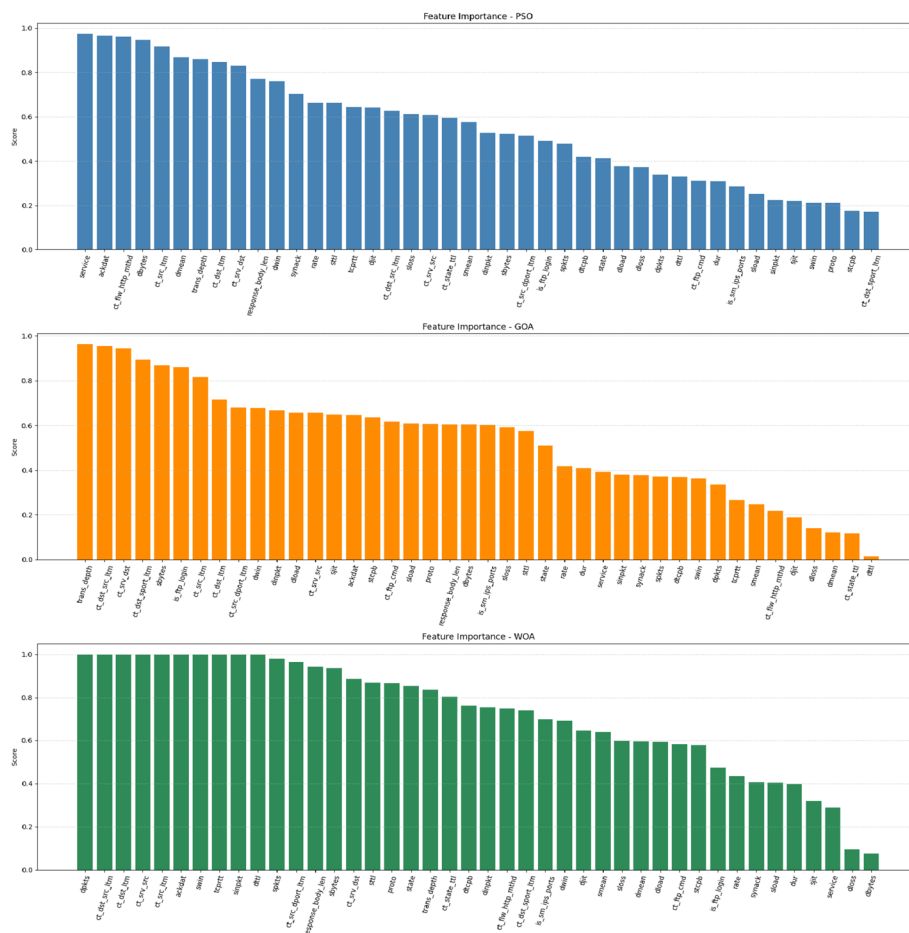
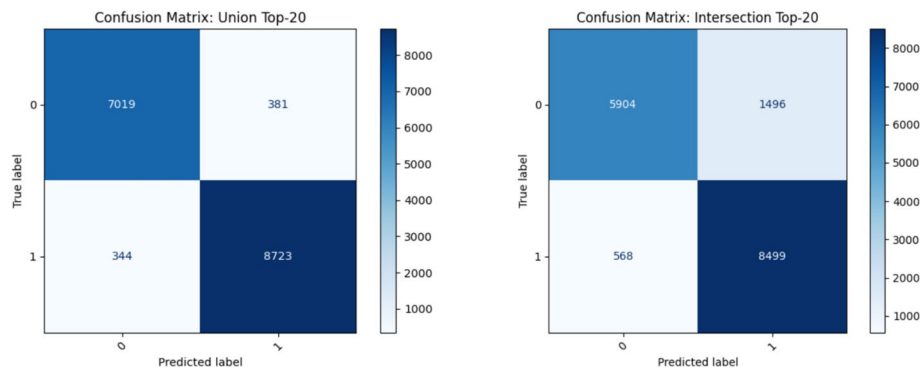


Fig.15 Feature importance for PSO, GOA and WOA methods

Table 6 Performance of the union and intersection approaches

Feature model	Accuracy	Precision	Recall	F1-score
Union (PSO, GO, WOA)	0.9560	0.9582	0.9621	0.9601
Intersection (PSO, GO, WOA)	0.8747	0.8503	0.9374	0.8917

**Fig. 16** Impact of feature selection on binary classification

importance, suggesting limited discriminative contribution and potential redundancy. Among the optimization techniques, PSO exhibits a sharper decline in importance scores, emphasizing a more aggressive feature reduction strategy, while GOA provides a more balanced distribution. WOA, on the other hand, assigns consistently high scores to a broader subset of features, reflecting its exploratory capability in preserving feature diversity. Overall, these results confirm that metaheuristic-based feature selection effectively reduces dimensionality while retaining the most informative traffic characteristics. This contributes to improved model efficiency, faster convergence, and enhanced generalization, particularly in federated and transfer learning–based intrusion detection settings.

5.2.2 Proposed feature selection method

The suitability of the proposed metaheuristic-based feature selection method was assessed using a base CNN model to evaluate the performance of each ensemble configuration on the UNSW-NB15 dataset, which includes 42 features and ten classes, as presented in Table 6.

Accordingly, the union-based approach consistently outperforms the intersection-based approach across all evaluation metrics, achieving an accuracy of 0.9560, precision of 0.9582, recall of 0.9621, and an F1-score of 0.9601. This superior performance can be attributed to the preservation of complementary and diverse feature subsets identified by different optimization algorithms. In contrast, the intersection-based approach exhibits a noticeable performance decline, with accuracy, precision, recall, and F1-score of 0.8747, 0.8503, 0.9374, and 0.8917, respectively. Although recall remains relatively high, the reduced precision and overall accuracy indicate that excessive feature elimination limits the model's discriminative capacity, resulting in inferior classification performance.

In the same line, we use a confusion matrix to evaluate how well the classification model performs using the selected features, as illustrated in Fig. 16.

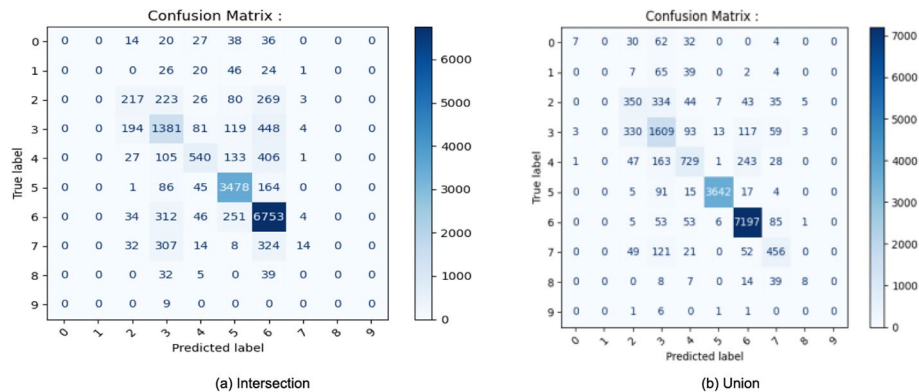


Fig. 17 Impact of feature selection on multi-class classifier

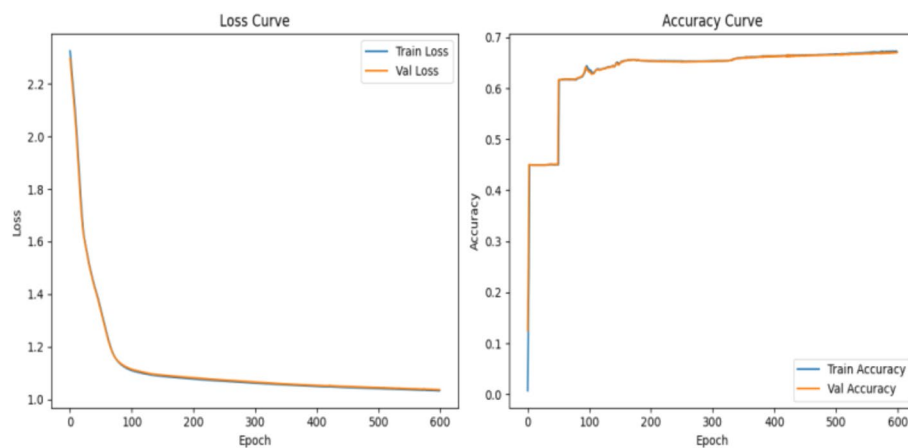


Fig. 18 Learning curves of basic CNN model

The simulation results demonstrate that the union approach consistently outperforms the intersection approach in binary classification tasks. This performance gain can be attributed to the union strategy's ability to preserve a richer and more informative feature set, thereby capturing a broader range of discriminative patterns in the data.

In multi-class classification, the confusion matrix illustrates the number of correct classifications and misclassifications for each class. The intersection approach, although more selective, may discard complementary features that are essential for accurate class separation, thereby resulting in comparatively reduced classification performance, as shown in Fig. 17.

Moreover, the learning dynamics and performance improvement of the CNN models over time are assessed using accuracy and loss curves, which are essential for evaluating model behavior and diagnosing potential issues. For the union approach, the learning curve, expressed in terms of loss or accuracy, illustrates the training performance of the CNN model, as shown in Fig. 18.

According to the simulation findings, the training and validation curves exhibit steady and well-behaved learning, with both losses gradually decreasing and closely overlapping throughout training, indicating good generalization and no signs of overfitting. Similarly, the training and validation accuracy curves follow nearly identical trajectories,

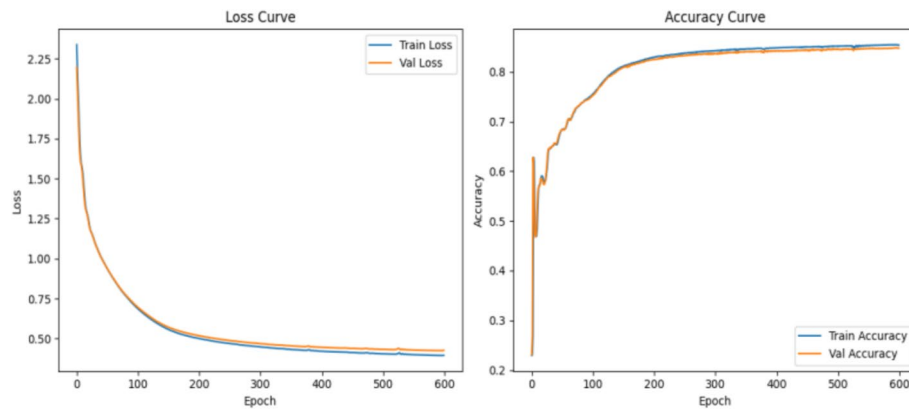


Fig. 19 Learning curves of the CNN model with feature selections

Table 7 Performance comparison of FedAVG and other models

Models	Accuracy	Precision	Recall	F1-score
Base CNN	0.9593	0.9700	0.9557	0.9628
SVM	0.6946	0.7247	0.6946	0.6727
KNN	0.8086	0.8091	0.8086	0.8088
FedAVG	0.9606	0.9686	0.9595	0.9640

showing rapid improvement during the initial epochs before converging to a stable plateau in the range of 66–68%.

In this work, we investigate whether the feature selection approach improves the learning behavior of the proposed IDS model. Accordingly, the learning curve is generated by plotting the error (loss) and accuracy values against the number of epochs for the union-based feature selection approach, as shown in Fig. 19.

The results show that the loss curves stabilize after an initial rapid decrease, indicating convergence of the training process for both the union and intersection approaches. The continuous reduction in training and validation losses, along with the minimal gap between them, demonstrates good model fitting without evidence of underfitting or overfitting. Furthermore, the accuracy curves of the CNN model, evaluated over 600 training epochs using the most important selected features, clearly highlight the superiority of the union approach, which consistently achieves higher accuracy by retaining more relevant and informative features for building and training the neural network in data center environments.

5.2.3 Federated averaging (FedAvg)

In this study, FedAvg is employed as a communication-efficient algorithm for distributed training across 10 clients. For binary classification, the simulation results demonstrate that the proposed FedAvg-based approach effectively improves detection accuracy while reducing the error rate in data center environments. In addition, the performance of various machine learning classifiers is evaluated on the UNSW-NB15 dataset. Table 7 summarizes the comparative performance of the different AI-based models.

Moreover, the confusion matrix is employed to comprehensively evaluate the binary classification performance of both the base CNN model and the proposed FedAvg-based model, as illustrated in Fig. 20. This analysis provides detailed insight into the data distribution, enabling a deeper assessment of each model's classification behavior, particularly

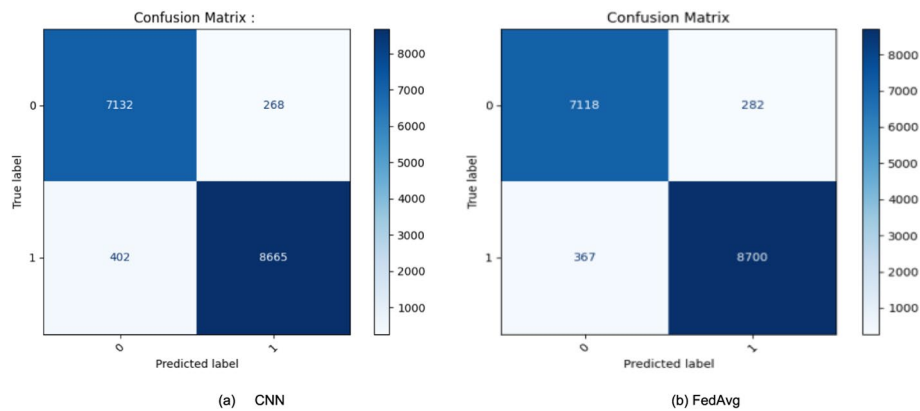


Fig. 20 Confusion matrix for FedAvg and CNN in binary classification

Table 8 Performance comparison of FedAvg and other models

Model	Accuracy	Precision	Recall	F1-score
Base CNN	0.8501	0.8424	0.8014	0.8428
CNN + Union	0.8534	0.8457	0.8534	0.8453
CNN + Intersection	0.7544	0.7178	0.7544	0.7210
FedAvg	0.8575	0.8539	0.8575	0.8549
SVM	0.6270	0.4851	0.6270	0.5203
KNN	0.6885	0.6800	0.6885	0.6807

in terms of correctly identifying true negative cases and reducing false positives. Consequently, the effectiveness of the proposed FedAvg approach in enhancing classification reliability and reducing misclassification rates compared to the baseline CNN model can be clearly observed.

The simulation results show that the proposed model outperforms existing techniques such as K-Nearest Neighbour (KNN), Support Vector Machine (SVM), and the base CNN model. The performance evaluation of the proposed TL-based model is presented in Table 8.

More importantly, the results demonstrate that feature selection can be as effective as traditional CNN learning, highlighting its ability to reduce computational time while simultaneously improving overall CNN performance.

Additionally, the proposed FedAvg-based IDS model is capable of distinguishing between multiple attack types. In this context, the per-class accuracy results reveal notable performance differences among the four evaluated models: FedAvg, the base CNN, CNN with union-based feature selection, and CNN with intersection-based feature selection. Overall, the superior generalization capability of FedAvg across decentralized data sources is demonstrated by its consistently achieving the highest accuracy across the majority of attack categories, as shown in Fig. 21.

Based on simulation results, FedAvg model achieves near-perfect classification accuracy for several classes, including Analysis, Backdoor, Fuzzers, Shellcode, and Worms, significantly outperforming the local CNN-based models. These results indicate that federated learning effectively leverages distributed knowledge and improves the model's capacity to identify subtle attack patterns that might not be fully represented at individual nodes. In contrast, the baseline CNN model exhibits moderate and inconsistent performance across classes. It suffers greatly on minority or complex classes such as

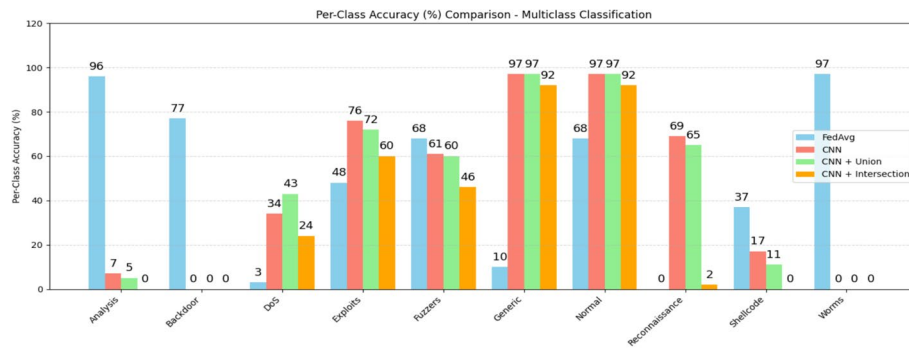


Fig. 21 Performance comparison of FedAvg and CNN model

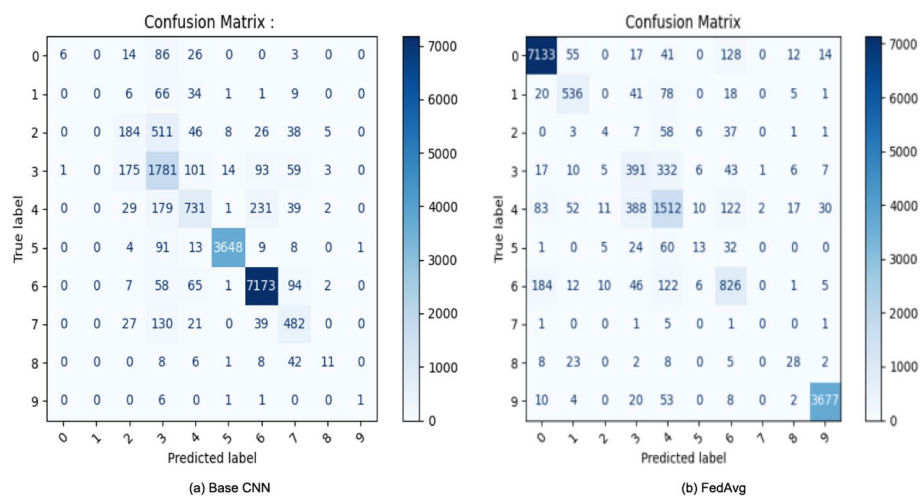


Fig. 22 Performance comparison for FedAvg and CNN in multi-class classification

Shellcode, Worms, and Backdoor, demonstrating its limitations when trained on centralized data with unbalanced distributions. Nevertheless, the CNN maintains relatively strong performance on dominant classes such as Generic and Normal traffic.

The CNN combined with the union-based feature selection strategy consistently outperforms the baseline CNN across several traffic categories, particularly DoS, Exploits, Generic, and Normal classes. This performance gain indicates that union-based feature selection improves generalization by preserving a more comprehensive set of informative features. In contrast, the CNN employing the intersection-based feature selection strategy generally demonstrates inferior performance. For most attack classes, the resulting reduced feature subset appears overly restrictive, leading to diminished classification accuracy. This suggests that the intersection-based approach may eliminate discriminative features that are crucial for effective classification, thereby adversely affecting overall model performance. Furthermore, a confusion matrix is used to compare the classification behavior of the CNN and FedAvg models on the same dataset, as illustrated in Fig. 22.

For advanced analysis, as illustrated in Fig. 23, the learning curve demonstrates a steady reduction in error throughout the training process until reaching a clear point of convergence. This convergence is characterized by a minimal gap between the training and validation losses, indicating stable learning behavior. These observations suggest

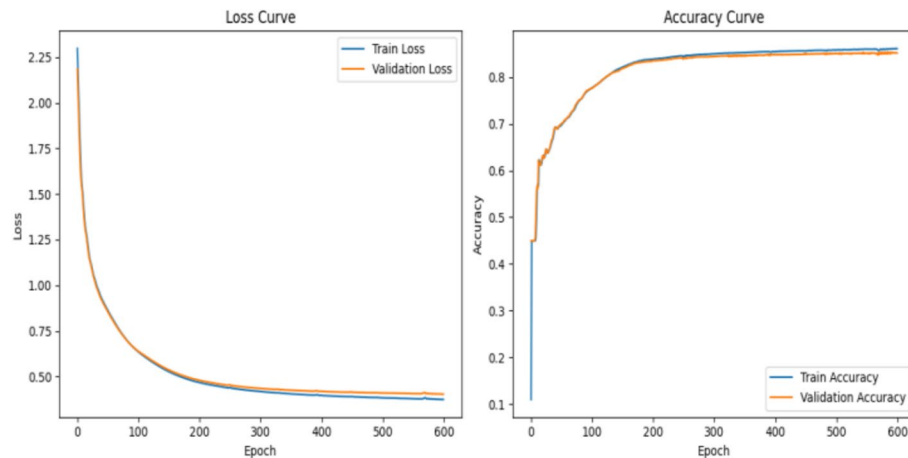


Fig. 23 Learning curves of the proposed FedAvg model

that the FedAvg model exhibits strong generalization capability, converges efficiently, and achieves improved performance without showing signs of overfitting or underfitting on the training dataset. These results highlight the robustness and stability of the FedAvg optimization process under the considered experimental settings.

Based on the simulation results, the proposed FedAvg model effectively detects multiple attack types on the UNSW-NB15 dataset, achieving high global accuracy. As a result, FedAvg consistently outperforms all other compared models across all evaluation metrics in distributed intrusion detection scenarios. As shown in Fig. 23, the learning curves indicate a good model fit, with training and validation losses decreasing steadily and converging to a stable point.

Despite the fact that non-IID data distributions typically hinder convergence and reduce model accuracy compared to IID settings, the experimental results demonstrate that the FedAvg-based approach remains robust. Accordingly, it achieves superior detection performance over centralized and traditional machine learning models. These outcomes therefore underscore the effectiveness of federated learning in mitigating data heterogeneity while maintaining high accuracy in distributed and privacy-sensitive environments.

5.3 Explainable deep learning

XAI methods aim to enhance the transparency of highly complex and opaque deep learning models by providing IDS models with reasoning that is both generalizable and understandable to humans. Among the most widely used techniques for improving interpretability in machine learning are SHAP and LIME. For deep learning models, SHAP offers a more comprehensive global interpretation, whereas LIME is particularly effective at providing local explanations. To achieve this, SHAP employs a mechanism known as the SHAP kernel to define coalition weights, ensuring that each coalition of features contributes appropriately to the estimation of Shapley values across the dataset. Essentially, SHAP unifies LIME and Shapley values by constraining explanations to an additive feature attribution model.

Formally, SHAP specifies the explanation model as follows:

$$\Upsilon(z') = \Phi_0 + \sum_{j=1}^{\mathbb{V}} \Phi_j z'_j, \quad (43)$$

where $\Upsilon(z')$ is the explanation model (an interpretable surrogate), $z' \in (z'_1, \dots, z'_V)^T \in \{0,1\}^{\mathbb{V}}$ is the coalition vector, Φ_j is the Shapley value for feature j , representing its contribution, Φ_0 is the expected model output (baseline), $\mathbb{V}$ is the maximum coalition size.

Subsequently, the linear model Υ is trained on the training dataset by minimizing the defined loss function, thereby learning the contribution of each feature by fitting the weighted samples generated by the SHAP kernel. To achieve Shapley-compliant weighting, Lundberg and Lee [54] derived the Kernel SHAP formulation for representing Shapley values as follows:

$$\Omega_x(z') = \frac{(\mathbb{V} - 1)}{\binom{\mathbb{V}}{|z'|} (V - |z'|)} \quad (44)$$

where $|z'|$ is the number of present features in instance z'

Moreover, we employ permutation feature importance (PFI) to verify the consistency of the SHAP contributions in identifying features that are essential to the prediction outcomes. Since our proposed framework uses a CNN model for both FedAvg and TL, we apply the proposed XAI methods to interpret the base CNN models using a dataset with the complete set of input features. The proposed XAI approach is adapted to evaluate CNN-based IDS models in edge data centers. First, a binary classification model is designed to detect attacks without identifying the specific attack type. Second, a one-vs-all classification model is developed to identify each attack class independently.

5.3.1 Binary classification

In this study, we use XAI models to assess the performance of a CNN-based binary classification model on the UNSW-NB15 dataset. Next, we apply SHAP and PFI methods to determine each feature's relevance score. The SHAP and PFI summary plots, shown in Fig. 24, reveal which UNSW-NB15 features have the greatest impact on the model's predictions and how their values influence the classification of network traffic as malicious or benign.

The most significant influence comes from the features at the top, including dttl, ct_state_ttl, ct_dst_src_ltm, and TCP window sizes. High values (red points) typically push SHAP values to the right, increasing the probability of a malicious classification, while low values (blue points) often shift predictions to the left, toward benign traffic. Accordingly, SHAP and PFI provide a ranking of the top- k most important features based on their average impact across all predictions. Specifically, we analyze the influence of these features for each attack type. As shown in Fig. 25, the horizontal spread reflects the strength with which each feature affects individual predictions, illustrating their impact on (a) attack traffic and (b) normal traffic.

The length of each bar reflects the magnitude of the feature's contribution, highlighting the relative influence of individual features on the final decision. The baseline value $E(f(X)) = 0.392$ represents the average model output.

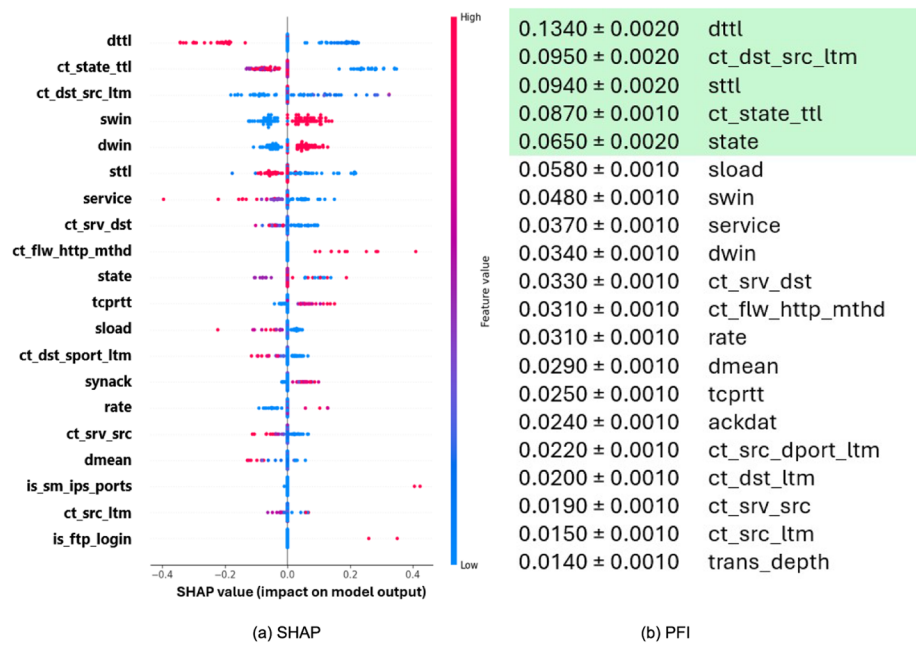


Fig. 24 Top 20 relevant features for binary classification

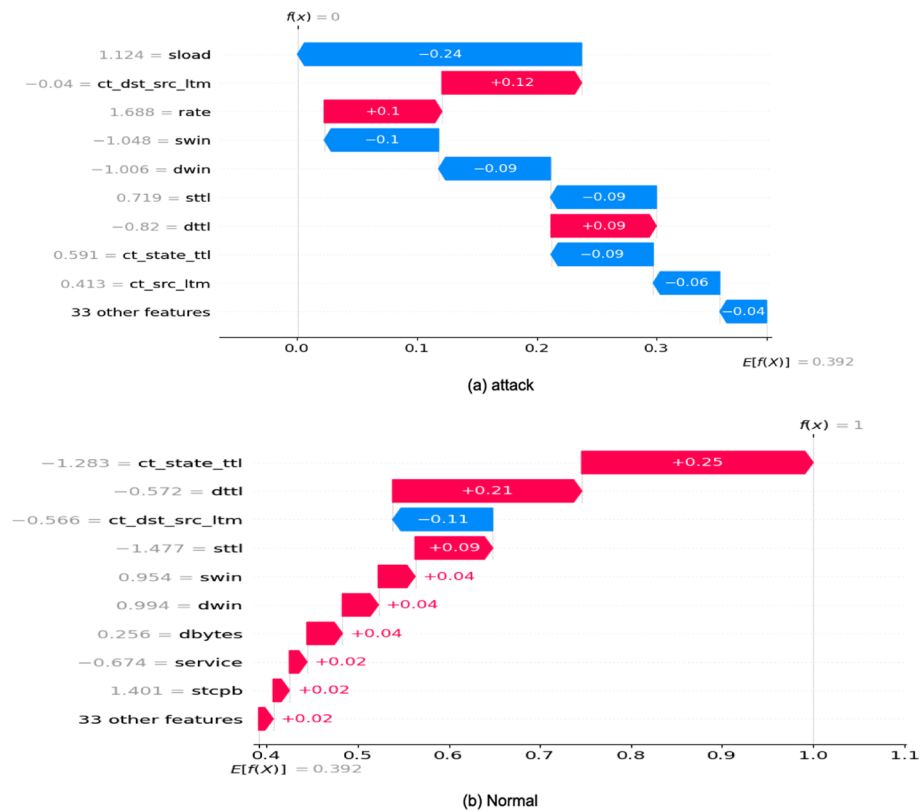


Fig. 25 Impact of features on each attack in binary classification

5.3.2 Multi-class classification

In this work, we propose a CNN-based multi-class intrusion detection system trained on the UNSW-NB15 dataset. By quantifying feature-level contributions, the proposed approach not only improves model transparency but also facilitates a deeper understanding of the underlying traffic patterns associated with malicious behavior. In this context, Fig. 26 presents an analysis of the model's decision-making process, highlighting attack class identification and the contribution of individual features through XAI-based interpretability methods.

The SHAP summary plot indicates that the intrusion detection system's (IDS) decision accuracy is driven primarily by a subset of features, notably `ct_srv_dst`, `ct_dst_src_ltm`, `sload`, and `ct_srv_src`, which demonstrate strong discriminatory power across multiple attack classes. This analysis provides valuable insight into the feature–class relationships that govern the model's classification decisions. High values of these features consistently push predictions toward the malicious class, thereby improving the model's ability to correctly identify attacks, whereas lower values tend to shift predictions toward benign behavior. In contrast, timing-related features such as `dwin`, `dmean`, and `sjit` exhibit substantially lower contributions. Furthermore, the dispersion of SHAP values highlights class-dependent behavior, revealing that the same feature may influence predictions differently depending on the attack type.

To gain deeper insight into the model's decision-making process, we employ SHAP waterfall plots to examine both the direction and magnitude of feature contributions for each attack type. These plots provide instance-level explanations by decomposing individual predictions into additive feature effects, highlighting how specific network traffic characteristics increase or decrease the likelihood of particular attack classes. As illustrated in Fig. 27, the horizontal span of each contribution reflects its influence on the final prediction, offering a transparent and intuitive explanation of class-dependent

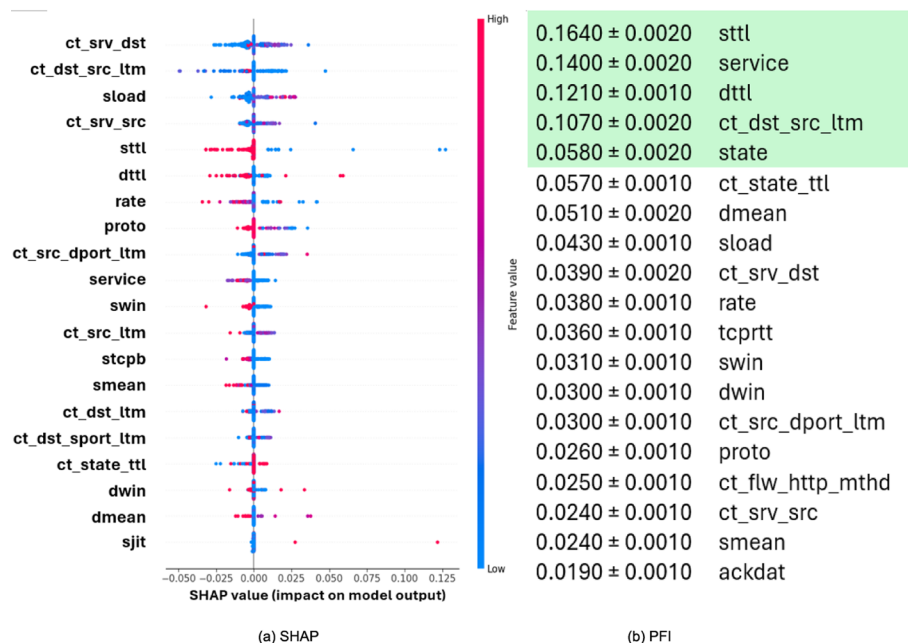


Fig. 26 Top 20 relevant features in multi classification

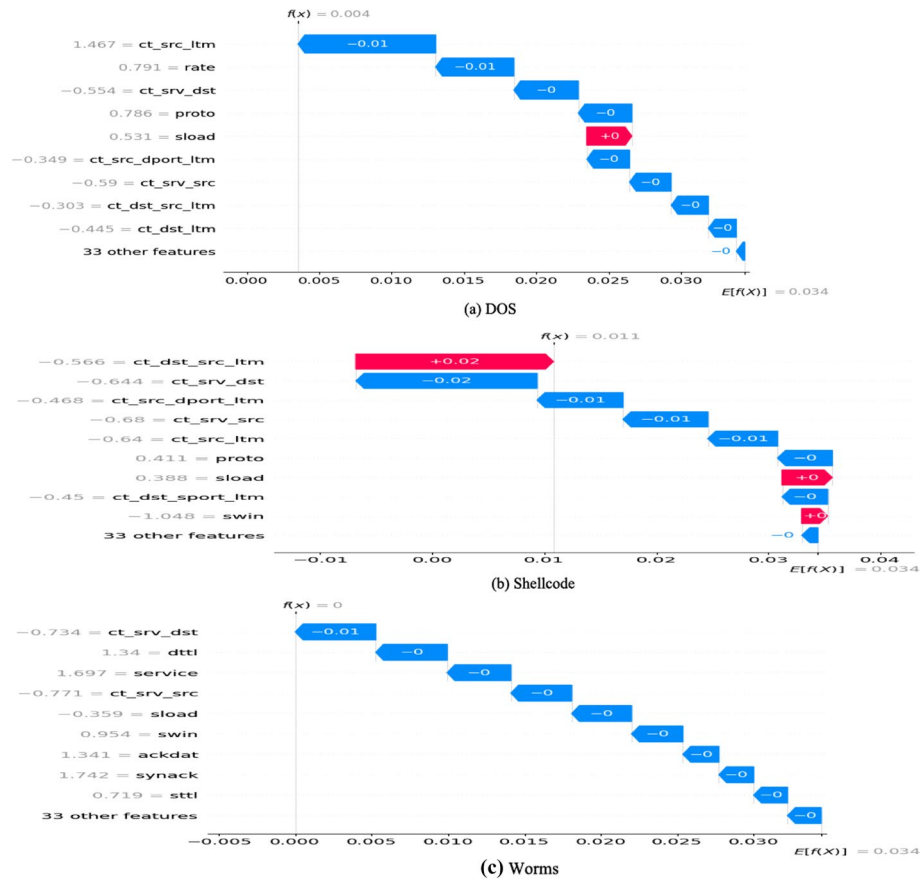


Fig. 27 Waterfall plot of SHAP values for features in the multi-class classification

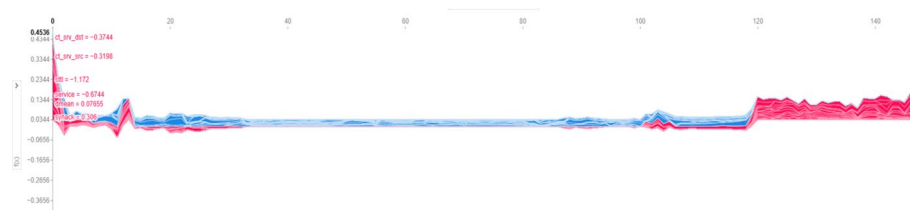


Fig. 28 Stacked Shapley values along clustered

behavior. Note that the bar length indicates feature strength, while the sign indicates whether the feature pushes the prediction toward a malicious or benign class.

Despite the computational overhead associated with SHAP, it is adopted as the core interpretability tool for analyzing the proposed IDS model, even in the presence of resource constraints on edge nodes. This choice is motivated by the need for reliable and consistent explanations in security-critical environments, where inaccurate or misleading interpretations may compromise trust and decision-making. By prioritizing SHAP, we aim to balance interpretability fidelity with deployment constraints, while leveraging PFI as a complementary method for cross-validation and high-level feature relevance analysis. After computing the Shapley values, clustering these SHAP values provides insights into the data by grouping samples with similar explanation patterns, as shown in Fig. 28. In this context, the figure depicts the features influencing the CNN model

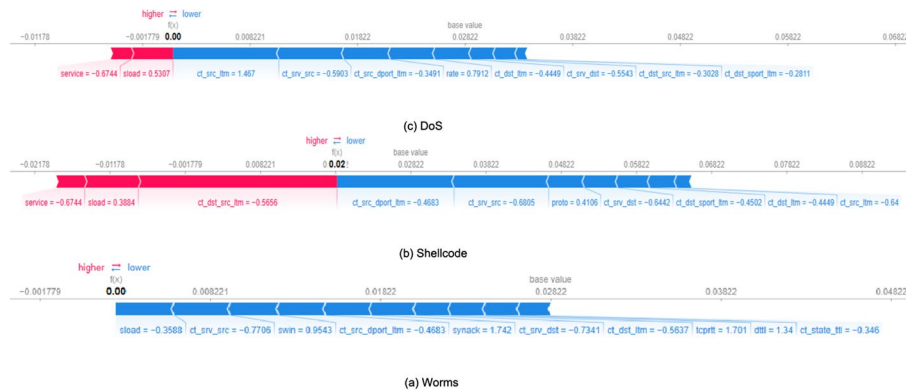


Fig. 29 Local explanations using SHAP force plot for each attack

for each attack in the UNSW-NB15 dataset, highlighting whether their contributions are lower (blue) or higher (red).

SHAP values were clustered to group samples with similar prediction explanations for the proposed IDS model, revealing distinct patterns of feature contributions that correspond to different model response behaviors. Observations are ordered along the x-axis according to hierarchical similarity, while the y-axis represents the aggregated Shapley values of the selected features. The plot reveals three distinct clusters: Cluster 1 (observations 0–18), Cluster 2 (observations 18–105), and Cluster 3 (observations 105–140), each characterized by unique contribution patterns reflecting different IDS model responses.

For the multi-class approach, the CNN model's decisions are decomposed into the aggregated contributions of individual feature values, enabling a clearer analysis of their impact on the predictions. We examine feature influences for three attack types, namely Shellcode, DoS, and Worms, with the results presented in Fig. 29.

For the DoS instance, *sload* and *service* emerge as the most influential features, indicating that traffic load intensity and service type play a critical role in driving the model's decision. In contrast, for the Shellcode attack, *ct_dst_src_ltm*, *sload*, and *service* contribute most significantly to the final prediction. This suggests that temporal connection patterns, traffic load intensity, and service characteristics collectively influence the model's classification of this attack type.

6 Conclusion

Cloud-edge data centers are expected to transform digital infrastructure by bringing data processing and storage closer to end devices, thereby addressing the growing demands for high performance and low latency. However, edge environments generally exhibit weaker security due to their limited memory and processing capabilities compared to hyperscale data centers. In next-generation security platforms, we explore how deep learning models can address intrusion detection challenges by identifying potential cyberattacks and analyzing malicious network traffic patterns, particularly for fine-tuning security configurations in next-generation data centers (NGDCs).

Within the proposed IDS framework, we implement both centralized and distributed detection models to construct federated learning (FL) and transfer learning (TL) architectures using CNN-based models. This hybrid approach, which integrates privacy-preserving learning with knowledge transfer, enables collaborative training across

distributed servers. In addition, we employ a fusion strategy to integrate dataset features as well as the outputs of three pre-trained CNN models. Simulation results demonstrate that the proposed framework achieves superior performance in both binary and multi-class classification tasks, thereby ensuring a more robust intrusion detection system for next-generation data centers (NGDCs).

To provide a more comprehensive and rigorous evaluation of the CNN-based models, particularly with respect to the features that most influence IDS performance, we incorporate SHAP and permutation feature importance (PFI) explainability techniques. SHAP is employed to interpret the contribution and relative significance of input features within the proposed IDS. The results highlight the effectiveness of the proposed framework, demonstrating how it successfully integrates the complementary strengths of multiple deep learning models and feature selection methods to enhance both performance and interpretability.

In future work, we plan to apply a Quantum Deep Autoencoder (QDL) to address anomaly detection in next-generation data centers (NGDCs). Owing to their reduced parameter requirements and fewer training iterations, QDLs offer the potential to design an IDS that enhances system robustness while accommodating the resource constraints of edge data centers. In parallel, we intend to integrate a Variational Autoencoder (VAE) for synthetic data generation to mitigate dataset imbalance. Additionally, we will investigate the impact of various non-IID data distributions on federated learning, with particular emphasis on effective data partitioning strategies. Although PFI and SHAP improve model interpretability, they face robustness challenges in noisy environments. Therefore, future research will focus on evaluating explanation stability, cross-client consistency, and the level of agreement among different explainable AI (XAI) methods.

Author contributions

Marwan M. developed the theory and wrote the main manuscript, Marwan M and CHEMMAKHA M. performed the simulations, AIT TEMGHART A. and LAZAAR M. verified the analytical methods. All authors read and approved the final manuscript.

Funding

The authors did not receive support from any organization for the submitted work.

Data availability

The data that support the findings of this study can be shared by the corresponding author upon reasonable request.

Declarations

Competing interests

The authors have no competing of interests to declare that are relevant to the content of this article.

Received: 15 December 2025 / Accepted: 2 February 2026

Published online: 24 March 2026

References

1. Caiazza C, Giordano S, Luconi V, Vecchio A. Chiara caiazza, silvia giordano, valerio luconi, alessio vecchio, Edge computing vs centralized cloud: impact of communication latency on the energy consumption of LTE terminal nodes. *Comput Commun.* 2022;194:213–25.
2. Edge AE, Abd Latiff MS, Chan WH. IoT data analytic algorithms on edge-cloud infrastructure: a review. *Digit Commun Networks.* 2023;9(6):1486–515.
3. Rong G, Xu Y, Tong X, et al. An edge-cloud collaborative computing platform for building AIoT applications efficiently. *J Cloud Comp.* 2021;10:36. <https://doi.org/10.1186/s13677-021-00250-w>.
4. Douch S, Abid MR, Zine-Dine K, Bouzidi D, Benhaddou D. Edge computing technology enablers: a systematic lecture study. *IEEE Access.* 2022;10:69264–302.
5. Deebak BD, AL-Turjman F. Lightweight authentication for IoT/Cloud-based forensics in intelligent data computing. *Future Gener Comput Syst.* 2021;116:406–25.

6. Sanjalawe Y, Anbar M, Al-E'mari S, Abdullah R, Hasbullah I, Aladaileh M. Cloud data center selection using a modified differential evolution. *Comput Mater Continua* 2021; 69: 3179–3204.
7. Khraisat A, Gondal I, Vamplew P, et al. Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity*. 2019;2:20.
8. Saha S, Hota A, Chattopadhyay AK, et al. A multifaceted survey on privacy preservation of federated learning: progress, challenges, and opportunities. *Artif Intell Rev*. 2024;57:184.
9. Nguyen TD, Marchal S, Miettinen M, Fereidooni H, Asokan N, Sadeghi A-R, Diot. A federated self-learning anomaly detection system for IoT. In: 2019 IEEE 39th International conference on distributed computing systems (ICDCS). IEEE; 2019. pp. 756–767.
10. Montavon G, Samek W, Müller KR. Methods for interpreting and understanding deep neural networks. *Digital Signal Process*. 2018;73:1–15.
11. Mohammadpour L, Ling TC, Liew CS, Aryanfar A. A Survey of CNN-based network intrusion detection. *Appl Sci*. 2022;12(16):8162.
12. Wu P, Guo H. LuNET: a deep neural network for network intrusion detection. In: Proceedings of the 2019 IEEE symposium series on computational intelligence (SSCI), Xiamen, China, 6–9 December 2019. pp. 617–624.
13. Zhang Y, Ran X. A step-based deep learning approach for network intrusion detection. *Comput Model Eng Sci*. 2021;128:1231–45.
14. Yin S, Zhang X, Liu S. Intrusion detection for capsule networks based on dual routing mechanism. *Comput Netw*. 2021;197:108328.
15. Lin P, Ye K, Xu C-Z. Dynamic network anomaly detection system by using deep learning techniques. In: Proceedings international conference cloud computer. Cham, Switzerland: Springer; 2019. pp. 161–176.
16. Shone N, Ngoc TN, Phai VD, Shi Q. A deep learning approach to network intrusion detection. *IEEE Trans Emerg Top Comput Intell*. 2018;2(1):41–50.
17. Li X, Hu Z, Xu M, Wang Y, Ma J. Transfer learning based intrusion detection scheme for Internet of vehicles. *Inf Sci*. 2021;547:119135.
18. Okey OD, Melgarejo DC, Saadi M, Rosa RL, Kleinschmidt JH, Rodríguez DZ. Transfer learning approach to IDS on cloud IoT devices using optimized CNN. *IEEE Access*. 2023;11:1023–38.
19. Li Z, Qin Z, Huang K, Yang X, Ye S. Intrusion detection using convolutional neural networks for representation learning, Neural information processing. *ICONIP 2017, Lecture notes in computer science*, vol 10638, Cham: Springer; 2017. pp. 858–866.
20. Zhu Y, Brettin T, Xia F, et al. Converting tabular data into images for deep learning with convolutional neural networks. *Sci Rep*. 2021;11:11325. <https://doi.org/10.1038/s41598-021-90923-y>.
21. Kumar H, Kumar H, Harish, Nandanwar H, Katarya R. Enhancing security and scalability of IoT systems using blockchain: addressing key challenges and limitations. In: Proceedings of the 6th international conference on deep learning, artificial intelligence and robotics (ICDLAIR 2024), Atlantis Press; 2025. https://doi.org/10.2991/978-94-6463-740-3_17
22. Nandanwar H, Katarya R. Optimized intrusion detection and secure data management in IoT networks using GAO-Xgboost and ECC-integrated blockchain framework. *Knowl Inf Syst*. 2025;67:9531–86. <https://doi.org/10.1007/s10115-025-02513-3>.
23. Nandanwar H, Katarya R. A hybrid Blockchain-Based framework for securing intrusion detection systems in internet of things. *Cluster Comput*. 2025;28:471. <https://doi.org/10.1007/s10586-025-05135-0>.
24. Nandanwar H, Katarya R (2025e) A secure and privacy-preserving IDS for IoT networks using hybrid blockchain and federated learning. In: Sharma DK, Sharma R, Peng SL (eds) Proceedings of international conference on next-generation communication and computing. NGCCOM 2024. Lecture notes in networks and systems, vol 1305. Springer, Singapore. https://doi.org/10.1007/978-981-96-3725-6_18
25. Nandanwar H, Katarya R. Privacy-preserving data sharing in blockchain-enabled IoT healthcare management system. *Comput J*. 2025;68(11):1657–81. <https://doi.org/10.1093/comjnl/bxaf065>.
26. Nandanwar H, Katarya R. Secure and privacy-preserving data sharing in 6G-enabled blockchain IoT healthcare systems. *Secur Priv*. 2025;8(6):e70105.
27. Wang M, Zheng K, Yang Y, Wang X. An explainable machine learning framework for intrusion detection systems. *IEEE Access*. 2020;8:73127–41.
28. Swana EF, Doorsamy W, Bokoro P. Tomek Link and SMOTE approaches for machine fault classification with an imbalanced dataset. *Sensors*. 2022;22:3246. <https://doi.org/10.3390/s22093246>.
29. Elreedy D, Atiya AF, Kamalov F. A theoretical distribution analysis of synthetic minority oversampling technique (SMOTE) for imbalanced learning. *Mach Learn*. 2024;113:4903–23.
30. Pereira RM, Costa YMG, Silla CN Jr. MLTL: a multi-label approach for the Tomek Link undersampling algorithm. *Neurocomputing*. 2020;383:95–105.
31. Kennedy J, Eberhart R. Particle swarm optimization (psa). In: Proceedings of IEEE international conference on neural networks, Perth, Australia; 1995. pp 1942–1948.
32. Mirjalili S, Lewis A. The whale optimization algorithm. *Adv Eng Softw*. 2016;95:51–67.
33. Zhang Q, Gao H, Zhan ZH, Junqing L, Zhang H. Growth optimizer: a powerful metaheuristic algorithm for solving continuous and discrete global optimization problems. *Knowl-Based Syst*. 2023;261:110206.
34. Shukla A, Chourasia K, Jain G, et al. DeepInsight: a CNN-based approach for machine reading comprehension in query answering systems and its applications. *Multimed Tools Appl*. 2024;83:3313–33. <https://doi.org/10.1007/s11042-023-17732-5>.
35. Deng J, Dong W, Socher R, Li L-J, Li K, Fei-Fei L. ImageNet: a large-scale hierarchical image database. In: 2009 IEEE conference on computer vision and pattern recognition, Miami, FL, USA; 2009. pp. 248–255.
36. Abdulkadrov R, Lyakhov P, Nagornov N. Survey of optimization algorithms in modern neural networks. *Mathematics*. 2023;11(11):2466.
37. Kingma DP, Ba J. Adam: a method for stochastic optimization. In: Proceedings of 3rd international conference on learning representations; 2015.
38. Bergstra J, Rémi Bardenet, Yoshua Bengio, Balázs Kégl, Algorithms for hyper-parameter optimization, NIPS'11: Proceedings of the 25th international conference on neural information processing systems; 2011. pp. 2546–2554.

39. Iqbal T, Wani MA. Weighted ensemble model for image classification. *Int J Inf Technol*. 2023;15:557–64. <https://doi.org/10.1007/s41870-022-01149-8>.
40. Zhao R, Mu Y, Zou L, Wen X. A hybrid intrusion detection system based on feature selection and weighted stacking classifier. *IEEE Access*. 2022;10:71414–26.
41. Cohen J. A coefficient of agreement for nominal scales. *Educ Psychol Meas*. 1960;20:37–46.
42. Cohen J. Weighted kappa: nominal scale agreement with provision for scaled disagreement or partial credit. *Psychol Bull*. 1968;70:213–20.
43. Zhu Q, Guo X, Deng W, Shi S, Guan Q, Zhong Y, et al. Land-use/land-cover change detection based on a Siamese global learning framework for high spatial resolution remote sensing imagery. *ISPRS J Photogramm Remote Sens*. 2022;184:63–78.
44. Efthymiadis F, Karras A, Karras C, Sioutas S. Advanced optimization techniques for federated learning on non-IID data. *Future Internet*. 2024;16(10):370.
45. Nanayakkara SI, Pokhrel SR, Li G. Understanding global aggregation and optimization of federated learning. *Future Gener Comput Syst*. 2024;159:114–33. <https://doi.org/10.1016/j.future.2024.05.009>.
46. Hakkoum H, Idri A, Abnane I. Global and local interpretability techniques of supervised machine learning black box models for numerical medical data. *Eng Appl Artif Intell*. 2024;131:107829. <https://doi.org/10.1016/j.engappai.2023.107829>.
47. Chen H, Covert IC, Lundberg SM, et al. Algorithms to estimate Shapley value feature attributions. *Nat Mach Intell*. 2023;5:590–601. <https://doi.org/10.1038/s42256-023-00657-x>.
48. Castro J, Go'mez D, Tejada J. Polynomial calculation of the Shapley value based on sampling. *Comput Oper Res*. 2009;36(5):1726–30.
49. Ancona M, Oztireli C, Gross M. Explaining deep neural networks with a polynomial time algorithm for Shapley value approximation. In: *International conference on machine learning*, PMLR; 2019. pp 272–281.
50. Breiman L. Random forests. *Mach Learn*. 2001;45(1):5–32. <https://doi.org/10.1023/A:1010933404324>.
51. Fisher A, Rudin C, Dominici F. All models are wrong, but many are useful: learning a variable's importance by studying an entire class of prediction models simultaneously. *J Mach Learn Res*. 2019;20(177):1–81.
52. Bakır H, Ceviz Ö. Empirical enhancement of intrusion detection systems: a comprehensive approach with genetic algorithm-based hyperparameter tuning and hybrid feature selection. *Arab J Sci Eng*. 2024;49:13025–43.
53. Moustafa N, Slay J. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: *Military communications and information systems conference (MilCIS)*; 2015. IEEE, pp. 1–6.
54. Lundberg SM, Lee SI. A unified approach to interpreting model predictions. In: *Proceedings of the 31st international conference on neural information processing systems (NIPS'17)*. Red Hook, NY: Curran Associates Inc.; 2017. pp. 4768–77.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.